

The numodel package*

Paul Zuurbier
mail@paulzuurbier.nl

May 19, 2026

Abstract

A LuaLaTeX package for writing and rendering numerical models (Euler-integrated dynamical systems) directly inside LaTeX documents, aimed at physics teaching material. It provides a text-model pipeline (`\mvar`, `\mrule`, `\computemodel`), Forrester stock-and-flow diagrams, and optional plots of the computed time series via the sibling package `numodel-plot`.

Contents

1	Introduction	1
2	First example: a free-falling ball	2
2.1	<code>\textmodel</code> — rule table	2
2.2	<code>\graphicmodel</code> — Forrester diagram	2
2.3	<code>\computemodel</code> + <code>\diagrammodel</code> — numerical plot	3
3	Configuration	3
3.1	Diagram styles in practice	4
4	Public API	5
4.1	Variables and rules	5
4.2	Expression reference	6
4.3	Render commands	8
4.4	Series accessors	8
4.5	Namespace management	9
5	Variable types	9
6	Generated accessors	9
7	Multiple models in one document	10
8	Requirements	10
9	Implementation	10
9.1	Translation files	49
9.1.1	<code>numodel-EN.def</code> — English (XMILE)	49
9.1.2	<code>numodel-NL.def</code> — Dutch (CoachTaal)	50

1 Introduction

`numodel` lets an author write a dynamical system (stocks, flows, helper variables, rules, and a stop condition) as a sequence of LaTeX macros and renders three complementary views of that model directly in the document:

- a *text model* — a typeset rule table with initial values (`\textmodel`);
- a *graphic model* — a Forrester stock-and-flow diagram with auto-layout (`\graphicmodel`);

*This document corresponds to `numodel` v0.3.0, dated May 19, 2026.

- a *diagram* — a numerical Euler simulation plus PGFPlot of any pair of variables (`\computemodel` followed by `\diagrammodel`; the plot is rendered through the sibling package `numodel-plot`).

All three views are produced from a single set of declarations so the textbook description, the conceptual stock-and-flow diagram, and the numerical result of the same model are guaranteed to stay in sync. Variables and rules live in namespaces (*prefixes*) so a document can contain multiple independent models; `\newmodelprefix{P}` starts a fresh one. The simulation engine runs in Lua (through `luacode`) for $\mathcal{O}(1)$ appends and cheap min/max tracking; the rendering layer is pure `expl3`.

2 First example: a free-falling ball

A ball dropped from $h_0 = 100$ m under constant gravitational acceleration. The complete model:

```
\usepackage[syntax=EN]{numodel}

\newmodelprefix{ball}
\mvar{T}{t}{0}{\s}{2}{system}
\mvar{Dt}{dt}{0.1}{\s}{2}{system}
\mvar{V}{v}{0}{\m\per\s}{2}{stock}
\mvar{Y}{y}{100}{\m}{3}{stock}
\mvar{G}{g}{-9.81}{\m\per\s\squared}{3}{aux}

\mrule{V}{\ballV + \ballG * \ballDt}
\mrule{Y}{\ballY + \ballV * \ballDt}
\mrule{T}{\ballT + \ballDt}
\mstop{\ballY <= 0}
```

After the declarations above, three render commands produce the three views shown below, each from the *same* model.

2.1 `\textmodel` — rule table

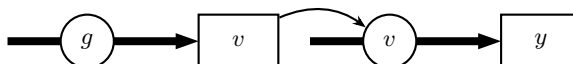
The verbatim source rendered by `\textmodel`:

	model	initial values
1	$v = v + g \cdot dt$	$t = 0$ s
2	$y = y + v \cdot dt$	$dt = 0.10$ s
3	$t = t + dt$	$v = 0$ m s ⁻¹
4	IF $y \leq 0$ THEN STOP ENDIF	$y = 100$ m $g = -9.81$ m s ⁻²

Each `\mvar` with a non-empty start value contributes a row in the *initial values* column; each `\mrule` contributes a row in the *model* column. Symbols come from the second `\mvar` argument (the display text), values are formatted through `siunitx`. `<=` is rendered as $\leq$.

2.2 `\graphicmodel` — Forrester diagram

`\graphicmodel` draws the same model as a stock-and-flow diagram. Stocks (type `stock`) are rectangles, constants (`constant`) are circles, helpers (`aux`) are unboxed identifiers; flows are inferred from rule structure (`\<stock> + ...` or `... + \<stock>` on the right-hand side):



Layout is automatic from the rule graph. Manual placement is available through `gridx/gridy` keys on the `\mvar` call (see Section 4). Wide diagrams can be capped to a maximum number of grid columns with `\numodelsetup{gridmaxx=N}`: when a row reaches N, the auto layout shifts the affected items up one row and continues filling.

2.3 `\computemodel` + `\diagrammodel` — numerical plot

`\computemodel` iterates the rules forward in time using Euler integration with step size `\ballDt`, stopping when `\mstop`'s condition becomes true (or when the `maxiter` safety limit is reached, see Section 3). `\diagrammodel{xvar}{yvar}{label}` then plots one variable against another:

```
\computemodel
\diagrammodel{T}{Y}{ball-fall}
```

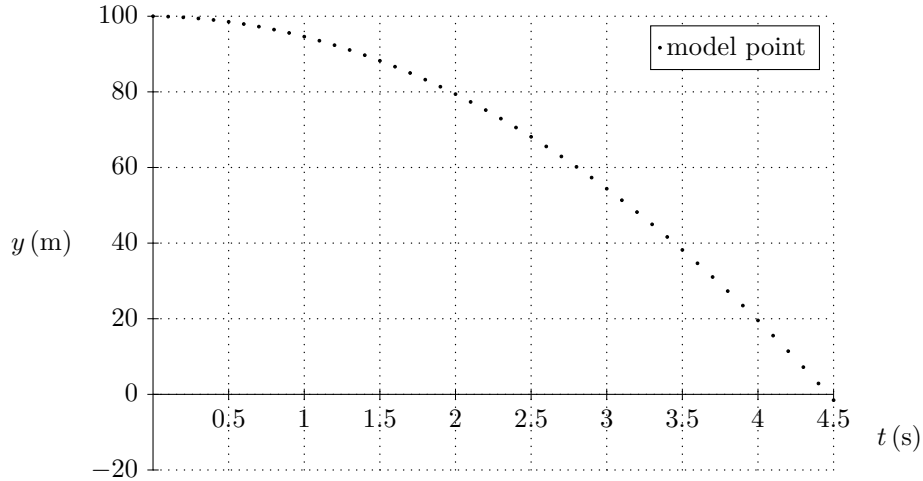


Figure 1: $y(t)$ -diagram

Axis ranges, tick lattice, and labels are computed automatically from the simulated min/max of each variable; the plot inherits the `numodel-plot` style (see that package's documentation for configuration).

3 Configuration

`\numodelsetup` Runtime configuration:

```
\numodelsetup{syntax=NL, maxiter=50000}
```

The same keys can also be passed as package options: `\usepackage[syntax=NL]{numodel}`. Recognised keys:

syntax Language tag for the rule-table rendering. Built-in values:

EN (default) XMLE-style ALL-CAPS keywords: IF/THEN/ELSE, AND, OR, ABS, SIGN, ...

NL Dutch CoachTaal keywords: Als/Dan/Anders, EN, OF, Abs, Teken, ...

The legacy names `english`, `coachtaal` and `dutch` are accepted as aliases for EN and NL. Each language tag `X` corresponds to a file `numodel-X.def` located via `kpse` when the package processes the key. The package ships with `numodel-EN.def` and `numodel-NL.def`; drop your own `numodel-FR.def` (or any other tag) in `TEXMFHOME/tex/latex/numodel/` and select it with `\usepackage[syntax=FR]{numodel}` – no package rebuild needed. The setting affects display only; the expression syntax in `\mrule` bodies is always `\fp_eval-compatible`.

maxiter Safety limit on the number of `\computemodel` iterations (default 20 000). When reached, the simulation aborts with a warning naming the unmet stop condition.

graphscalex Horizontal grid spacing in centimetres for `\graphicmodel`'s Forrester layout (default 2). Larger values spread the diagram out horizontally.

graphscaley Vertical grid spacing in centimetres for `\graphicmodel`'s Forrester layout (default 2). Larger values spread the diagram out vertically.

stockwidth Half-width of stock rectangles in `\graphicmodel` (default 0.375).

gridmaxx Maximum number of grid columns the auto layout may fill on any one row before wrapping (integer, default 0 = no limit). When the limit is reached, items already placed on the affected row (and everything above it for the stocks row, everything but stocks for the aux row, only the constants for the constants row) shift up by one row to free space, and placement continues from column 0. Manually positioned variables (`\mvar[gridx=...,gridy=...]`) are kept where they are. When wrapping is active the default centring of the aux row and the right-aligning of the stocks row are disabled, so the diagram fills left-to-right, bottom-to-top.

diagram-style Rendering style for the case where a helper or constant is the direct inflow/outflow of a stock. Three values:

tight (default) the valve takes the helper's/constant's label; the helper/constant itself is not drawn as a separate node. Compact and most LaTeX-native.

forrester Forrester/Sterman convention: the valve is drawn without a label and the helper/constant remains as a separate node connected to the valve by a causal arrow.

edu Didactic dual form: the valve carries the label *and* the helper/constant is drawn as a separate node with a causal arrow to the valve. Visually busy but pedagogically explicit.

flowarrow-style Visual style of the flow pipe. **hollow** renders the classic Forrester double-line pipe with an open arrow head; **filled** renders a thick solid arrow. The default tracks **diagram-style**: **forrester** picks **hollow**, the other styles pick **filled**. An explicit value overrides this coupling.

valve-style Visual style of the valve node. **valve** draws the bow-tie/butterfly icon (Forrester); **circle** draws an empty circle on the flow pipe; **edu** draws a labelled circle (the flow variable's display text inside). The default tracks **diagram-style**: **forrester** picks **valve**, the other styles pick **edu**.

flowarrow-cloud-tip Whether the open end of an inflow or outflow pipe is anchored to a cloud node, signalling the model boundary. Default tracks **diagram-style**: **forrester** picks **true**, the other styles pick **false**. May be set globally via `\numodelsetup`, per-render via `\graphicmodel`, or per-stock via `\mvar[flowarrow-cloud-tip=...]`. The most specific source wins.

units Whether the *initial values* cells in `\textmodel` display the SI unit alongside the value (`\qty`) or only the numeric value (`\num`). Boolean, default **true**. May also be supplied to `\textmodel[units=false]` as a per-table override; the global setting is restored after rendering.

decimal-separator Decimal mark used by every number that `numodel` renders: the *initial values* column of `\textmodel`, and the tick labels of `\diagrammodel`. Two values:

comma use a comma (siunitx output-decimal-marker={,}, pgfplots/pgf/number format/use comma).

point use a full stop (siunitx output-decimal-marker={.}, pgfplots/pgf/number format/use period).

The default tracks **syntax**: NL picks **comma**, EN picks **point**. Other language files can publish a default by defining `\_numodel_kw_<LANG>dsep_default`: (expanding to **point** or **comma**); when the macro is absent the default is **point**. An explicit **decimal-separator** key locks that choice and overrides any future **syntax** change. The override is scoped: `numodel` applies it only inside its own renderers (siunitx's state is restored on group exit), so a document-wide `\sisetup` is not perturbed.

3.1 Diagram styles in practice

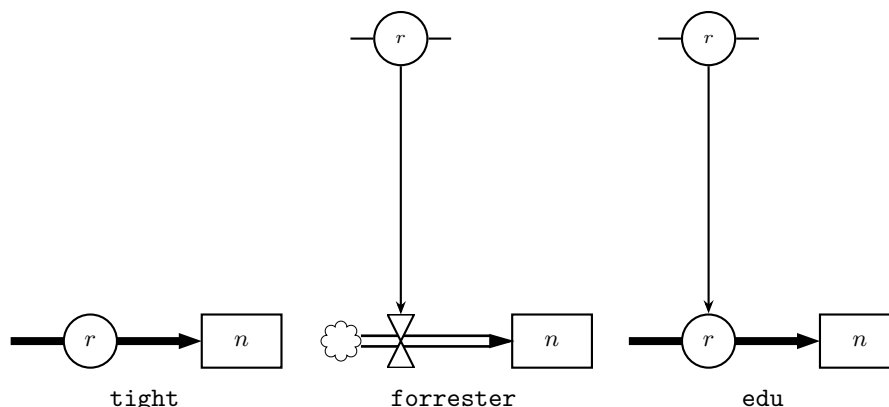
The same model rendered under each of the three **diagram-style** values. The model is the simplest case that distinguishes the styles: one stock N with a constant inflow R :

```
\newmodelprefix{flux}
\mvar{T}{t}{0}{\s}{2}{system}
\mvar{Dt}{dt}{1}{\s}{2}{system}
\mvar{N}{n}{0}{\s}{0}{stock}
\mvar{R}{r}{5}{\per\s}{2}{constant}
\mrule{N}{\fluxN + \fluxR * \fluxDt}
```

```

\mrule{T}{\fluxT + \fluxDt}
\mstop{\fluxT >= 5}
\graphicmodel[diagram-style=tight]
\graphicmodel[diagram-style=forrester]
\graphicmodel[diagram-style=edu]

```



- **tight** collapses the constant R into the valve label, producing the most compact diagram.
- **forrester** keeps the canonical System-Dynamics convention: unlabelled bow-tie valve, the constant remains a separate node, the link from R to the valve is a thin causal arrow.
- **edu** is a didactic dual: the valve carries the label *and* the constant remains as a separate node with a causal arrow. Less compact but pedagogically explicit – useful when first introducing the stock/flow vocabulary.

4 Public API

4.1 Variables and rules

\mvar Declares a model variable. Signature:

```
\mvar[<keys>]{<Name>}{<text>}{<start>}{<unit>}{<sig>}{<type>}
```

where $\langle Name \rangle$ is a short alphabetic identifier (the prefix-qualified accessor becomes $\backslash\langle prefix \rangle\langle Name \rangle$), $\langle text \rangle$ is the math-mode display symbol used in the rule table and diagram (e.g. F_{res}), $\langle start \rangle$ is the initial value (a number, or empty for helpers computed by a rule, or any `expl3` `fp`-evaluable expression involving previously defined model variables), $\langle unit \rangle$ is a bare `siunitx` unit macro sequence (e.g. $\backslash m\backslash per\backslash s\backslash squared$), $\langle sig \rangle$ is the number of significant figures used by $\backslash\langle prefix \rangle\langle Name \rangle num/qty$, and $\langle type \rangle$ is one of **stock**, **aux**, **constant**, or **system**. Each English type also accepts a Dutch alias (**voorraad**, **hulp**, **constante**, **systeem**) for backwards compatibility with existing teaching material. See Section 5.

Optional $\langle keys \rangle$:

prefix Override the current prefix for this single call.

gridx, **gridy** Manual placement in the `\graphicmodel` grid; integers, -1 leaves the slot to auto-layout (default).

alias Math-mode token list that replaces the entire *initial values* cell.

aliasleft, **aliasright** Replace just the left symbol or right value half of the cell.

\mrule Adds a rule of the form $\langle LHS \rangle \leftarrow \langle expr \rangle$. Signature:

```
\mrule*[<keys>]{<LHS>}{<expr>}
```

Both forms add the rule to *both* the rule table (`\textmodel`) and the simulation (`\computemodel`); execution is identical. The star only changes the typeset layout when $\langle expr \rangle$ is a ternary `cond ? a : b`. Without the star the ternary is rendered on a single table row,

```
IF cond THEN lhs = a ELSE lhs = b ENDIF
```

which is compact but wide. With the star (`\mrule*`) the same ternary is broken across rows,

```
IF cond THEN
  lhs = a
ELSE
  lhs = b
ENDIF
```

keeping the table column narrow so a `\graphicmodel` can sit alongside it, and making the source itself easier to read. For non-ternary expressions the star has no effect. $\langle expr \rangle$ may use the full `\fp_eval` expression grammar. See Table 1 (Section 4.2) for a complete overview of supported operators and functions with their XMILE and CoachTaal equivalents.

`\mruletext` Inserts a free-text row in the rule table without registering a rule with the simulator. Signature: `\mruletext[<keys>]{<text>}`. Useful for inserting comments or section dividers in long rule tables.

`\mstop` Sets the simulation stop condition. Signature: `\mstop[<keys>]{<expr>}`. The simulation halts at the first step where $\langle expr \rangle$ evaluates true. Exactly one `\mstop` per model prefix is required before `\computemodel`. Without one, `\computemodel` issues a warning.

4.2 Expression reference

Table 1 lists all expression constructs for `\mrule` bodies. The **XMILE** column shows XMILE v1.0 syntax, **l3fp** shows the `\fp_eval`-compatible form used inside `\mrule`, and **CoachTaal** shows the Coach 7 equivalent (function names from the standard CoachTaal math reference; note that argument separators in CoachTaal are semicolons). **Orange** entries are not yet fully supported by `numodel` (the expression computes correctly, but the rendered keyword in the rule table is not yet translated). *Italic* cells contain a derived equivalent rather than a native keyword.

Table 1: Expression reference: XMILE, `\fp_eval`, and CoachTaal

XMILE	l3fp (<code>\fp_eval</code>)	CoachTaal
<i>Arithmetic operators</i>		
+	+	+
-	-	-
*	*	*
/	/	/
^	^	^
MOD(x,y)	$x - \text{trunc}(x/y) * y$	$x - \text{Entier}(x/y) * y$
<i>Comparison operators</i>		
<	<	<
<=	<=	<=
>	>	>
>=	>=	>=
=	=	=
<>	!=	<>
<i>Boolean operators</i>		
AND	&&	EN
OR		OF
NOT	!	NIET
<i>Control flow</i>		
IF c THEN a ELSE b	c ? a : b	Als c Dan a Anders b EindAls
<i>General math functions</i>		
ABS(x)	abs(x)	Abs(x)
SIGN(x)	sign(x)	Teken(x)
SQRT(x)	sqrt(x)	Sqrt(x)
INT(x)	trunc(x)	$\text{Entier}(\text{Abs}(x)) * \text{Teken}(x)$
INT(x+0.5)	round(x)	Round(x)
INT(x)	floor(x)	Entier(x)

XMILE	l3fp (\fpeval)	CoachTaal
$-INT(-x)$	<code>ceil(x)</code>	$-Entier(-x)$
<code>MIN(x,y)</code>	<code>min(x,y)</code>	$Min(x;y)$
<code>MAX(x,y)</code>	<code>max(x,y)</code>	$Max(x;y)$
—	<code>fact(x)</code>	$Fac(x)$
$INT(LOG10(ABS(x)))$	<code>logb(x)</code>	$Entier(Log(Abs(x)))$
<i>Exponential and logarithmic</i>		
<code>EXP(x)</code>	<code>exp(x)</code>	$Exp(x)$
<code>LN(x)</code>	<code>ln(x)</code>	$Ln(x)$
<code>LOG10(x)</code>	$ln(x)/ln(10)$	$Log(x)$
<i>Trigonometry (radians)</i>		
<code>SIN(x)</code>	<code>sin(x)</code>	$Sin(x)$
<code>COS(x)</code>	<code>cos(x)</code>	$Cos(x)$
<code>TAN(x)</code>	<code>tan(x)</code>	$Tan(x)$
<code>ARCSIN(x)</code>	<code>asin(x)</code>	$Arcsin(x)$
<code>ARCCOS(x)</code>	<code>acos(x)</code>	$Arccos(x)$
<code>ARCTAN(x)</code>	<code>atan(x)</code>	$Arctan(x)$
<code>ARCTAN2(y,x)</code>	<code>atan(y,x)</code>	$Arctan(y/x)$
$1/TAN(x)$	<code>cot(x)</code>	$1/Tan(x)$
$1/SIN(x)$	<code>csc(x)</code>	$1/Sin(x)$
$1/COS(x)$	<code>sec(x)</code>	$1/Cos(x)$
$ARCSIN(1/x)$	<code>acsc(x)</code>	$Arcsin(1/x)$
$ARCCOS(1/x)$	<code>asec(x)</code>	$Arccos(1/x)$
$ARCTAN(1/x)$	<code>acot(x)</code>	$Arctan(1/x)$
$ARCTAN2(x,y)$	<code>acot(y,x)</code>	$Arctan(x/y)$
<i>Trigonometry (degrees)</i>		
$SIN(PI/180*x)$	<code>sind(x)</code>	$Sin(Pi/180*x)$
$COS(PI/180*x)$	<code>cosd(x)</code>	$Cos(Pi/180*x)$
$TAN(PI/180*x)$	<code>tand(x)</code>	$Tan(Pi/180*x)$
$180/PI*ARCSIN(x)$	<code>asind(x)</code>	$180/Pi*Arcsin(x)$
$180/PI*ARCCOS(x)$	<code>acosd(x)</code>	$180/Pi*Arccos(x)$
$180/PI*ARCTAN(x)$	<code>atand(x)</code>	$180/Pi*Arctan(x)$
$180/PI*ARCTAN2(y,x)$	<code>atand(y,x)</code>	$180/Pi*Arctan(y/x)$
$1/TAN(PI/180*x)$	<code>cotd(x)</code>	$1/Tan(Pi/180*x)$
$1/SIN(PI/180*x)$	<code>cscd(x)</code>	$1/Sin(Pi/180*x)$
$1/COS(PI/180*x)$	<code>secd(x)</code>	$1/Cos(Pi/180*x)$
$180/PI*ARCSIN(1/x)$	<code>acscd(x)</code>	$180/Pi*Arcsin(1/x)$
$180/PI*ARCCOS(1/x)$	<code>asecd(x)</code>	$180/Pi*Arccos(1/x)$
$180/PI*ARCTAN(1/x)$	<code>acotd(x)</code>	$180/Pi*Arctan(1/x)$
$180/PI*ARCTAN2(x,y)$	<code>acotd(y,x)</code>	$180/Pi*Arctan(x/y)$
<i>Constants</i>		
<code>PI</code>	<code>pi</code>	Pi
<code>e</code>	$exp(1)$	$Exp(1)$
<code>INF</code>	<code>inf</code>	—
<code>NAN</code>	<code>nan</code>	—
$PI/180$	<code>deg</code>	$Pi/180$
<code>1</code>	<code>true</code>	Aan
<code>0</code>	<code>false</code>	Uit
<i>Simulation-specific functions (XMILE only)</i>		
<code>TIME</code>	<i>user variable (\mvar)</i>	<i>user variable</i>
<code>DT</code>	<i>user variable (\mvar)</i>	<i>user variable</i>
<code>STEP(h,t₀)</code>	$T \geq t_0 \text{ ? } h : 0$	$Als\ T \geq t_0\ Dan\ h\ Anders\ 0\ EindAls$
<code>RAMP(s,t₀)</code>	$T > t_0 \text{ ? } s*(T-t_0) : 0$	$Als\ T > t_0\ Dan\ s*(T-t_0)\ Anders\ 0\ EindAls$
<code>DELAY(x,dt)</code>	<i>not supported</i>	<i>not supported</i>
<code>SMOOTH(x,t)</code>	<i>not supported</i>	<i>not supported</i>
<code>INIT(x)</code>	<i>not supported</i>	<i>not supported</i>

XMILE	l3fp (\fpeval)	CoachTaal
PREVIOUS(x)	<i>not supported</i>	<i>not supported</i>
RANDOM(0,1)	rand()	Rand
RANDOM(lo,hi)	$lo + (hi-lo)*rand()$	$lo + (hi-lo)*Rand$
<i>not supported</i>	randint(n)	<i>not supported</i>
<i>not supported</i>	randint(m,n)	<i>not supported</i>

4.3 Render commands

\textmodel	Renders the rule-and-startvalue table. Optional [<keys>] accepts <code>prefix=<name></code> (render a non-current model) and either <code>units=true</code> or <code>units=false</code> , which overrides the global <code>\numodelsetup</code> setting for this single render only. The global state is restored afterwards.
\graphicmodel	Renders the Forrester stock-and-flow diagram. Variables of type <code>stock</code> become rectangles, <code>constant</code> become circles, <code>aux</code> become identifier nodes; flow arrows connect stocks to constant or helper sources/sinks based on which variables appear in the right-hand side of stock-updating rules. Optional [<keys>] accepts <code>prefix=<name></code> (render a non-current model) and <code>diagram-style=tightforrestered</code> which overrides the global <code>\numodelsetup</code> setting for this single render only. The global state is restored afterwards, so multiple <code>\graphicmodel</code> calls can each pick their own style without re-issuing <code>\numodelsetup</code> .
\computemodel	Runs the Euler simulation in Lua. Records every variable's value at every step, plus running min and max. After it returns, the accessors <code>\<prefix><Name>min</code> / <code>\<prefix><Name>max</code> hold the extrema, and <code>\<prefix><Name></code> holds the final-step value. The time-series can be retrieved with <code>\mcoords</code> / <code>\mstep</code> .
\diagrammodel	Convenience wrapper that produces a complete figure with caption and label. Signature:

`\diagrammodel[<keys>]{<xvar>}{<yvar>}[<extra>]{<label>}`

Reads min/max and display text from `\<prefix><xvar>` etc., delegates to `\drawplot` from `numodel-plot`, and emits `\caption{$y(x)$-diagram}\label{fig:<label>}`. The optional `<extra>` argument is appended to the `axis` body, useful for additional `\addplot` lines (annotations, theoretical curves). Must be called after `\computemodel`.

4.4 Series accessors

\mcoords	Returns a comma-separated PGFPlots coordinate list of the simulated series for two variables. Fully expandable. Signature: <code>\mcoords{<xvar>}{<yvar>}</code> (current prefix); <code>\mcoordsp{<prefix>}{<xvar>}{<yvar>}</code> (explicit prefix). Both forms are usable inside <code>\addplot coordinates{ ... }</code> .
\mstep	Returns the value of one variable at a chosen iteration. Fully expandable. Signature: <code>\mstep{<Name>}{<i>}</code> ; <code>\mstepp{<prefix>}{<Name>}{<i>}</code> . The current prefix is prepended automatically by <code>\mstep</code> . Step indexing is 0-based: step 0 is the initial-values row, step $N-1$ is the final recorded step after <code>\computemodel</code> . Negative indices count from the end (Python-style), so <code>\mstep{Y}{-1}</code> is the last recorded y and <code>\mstep{Y}{-2}</code> is the penultimate one. Returns nothing (silently) if the index is out of range.

Example – a red secant line through the last two simulated (t, y) points of the free-fall ball (the model from the first example), extrapolated across the whole t -domain and labelled in the legend. The slope is the rise-over-run of the last two steps, the intercept is the final y -value:

```
\diagrammodel{T}{Y}{%
  \addplot[red, very thick, domain=\ballTmin:\ballTmax]
    { \mstep{Y}{-1}
      + ( \mstep{Y}{-1} - \mstep{Y}{-2} )
      / ( \mstep{T}{-1} - \mstep{T}{-2} )
      * ( x - \mstep{T}{-1} ) };
  \addlegendentry{secant endpoints}
} {ball-fall-secant}
```

Inside the optional [<extra>] argument `\mstep` expands as a literal number into PGFPlots' math parser, so each `\mstep` is evaluated only once (when the expression is constructed) rather than per sample. Applied to the `ball` model:

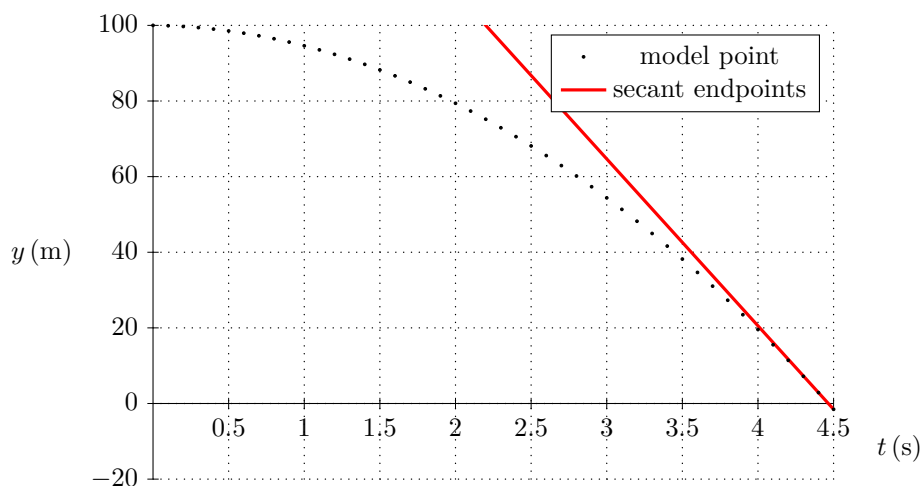


Figure 2: $y(t)$ -diagram

4.5 Namespace management

- `\newmodelprefix` Creates a new model namespace and switches to it. Signature: `\newmodelprefix{<name>}`. Subsequent `\mvar` / `\mrule` / `\mstop` calls bind to this prefix. All generated accessors are prefixed (so `\mvar{Y}{y}{...}{...}{...}{...}` under prefix `ball` produces `\ballY`, `\ballYtext`, etc.).
- `\switchmodelprefix` Switches to a previously created prefix. Useful when a document defines several models early and renders them later out of order. Signature: `\switchmodelprefix{<name>}`.
- `\NumodelForEachVar` Iterates a token list over every registered variable across every known prefix. Signature: `\NumodelForEachVar{<code with #1>}`; inside the body, `#1` is the full prefixed name (e.g. `ballY`). Used by external tools (e.g. a worksheet system) that need to expand every model accessor.

5 Variable types

The sixth `\mvar` argument (*<type>*) tags a variable with a role. The type drives both `\textmodel` layout (whether the row appears in *initial values*) and `\graphicmodel` node shape. Each type has a canonical English name and a Dutch alias; the two are interchangeable.

- stock** (alias *voorraad*) A stock that accumulates over time. Drawn as a rectangle in the Forrester diagram; its rule must be of the form `stock  $\leftarrow$  stock + ...` so that the integrator can detect inflows and outflows. Receives an *initial values* row.
- constant** (alias *constante*) A constant parameter (mass, gravitational acceleration, spring stiffness, ...). Drawn as a circle. Receives an *initial values* row.
- aux** (alias *hulp*) An auxiliary variable computed from other variables on each step. Drawn as a plain identifier node, no rectangle. No *initial values* row (start value is normally left empty).
- system** (alias *systeem*) System-level bookkeeping (time `T`, step size `Dt`, terminal time, ...). Drawn separately, no flow arrows. Receives an *initial values* row.

6 Generated accessors

Each `\mvar` call generates a family of accessor macros named `\<prefix><Name><suffix>`. The full set:

- (no suffix)** Current numeric value (post-rule-update if inside a step, post-final-step after `\computemodel`).
- text** The display symbol passed as the second argument of `\mvar`.
- unit** `\unit{...}` applied to the unit argument (siunitx-formatted).

unitraw The unit argument verbatim, without the `\unit{}` wrapper, suitable for use as a building block (e.g. `\xlabelunit` in `numodel-plot`).

num The current value formatted via `siunitx`'s `\num{}` with the variable's significant figures. Used by `\textmodel` in the *initial values* column when `units=false`.

qty As `num`, but including the unit (`\qty{value}{unit}`). Used by `\textmodel` in the *initial values* column when `units=true` (the default).

pre As `qty`, but with engineering prefix mode (e.g. `1500 W` renders as `1{,}5{kW}`).

sign Significant-figure count (raw integer).

type Variable type (raw string).

min, max Extrema over the simulated series. Empty before `\computemodel`.

gridx, gridy Manual placement coordinates in the Forrester grid; `-1` if left to auto-layout.

alias, aliasleft, aliasright Override tokens for the *startwaarden* cell layout.

7 Multiple models in one document

Each `\newmodelprefix` starts a fresh namespace. This lets a document contain several unrelated models without name clashes:

```
\newmodelprefix{ball}
\mvar{Y}{y}{100}{\m}{3}{stock}
% ... ball model ...

\newmodelprefix{spring}
\mvar{X}{x}{0.1}{\m}{3}{stock}
% ... spring model ...

% Render the ball model:
\switchmodelprefix{ball}\textmodel\computemodel\diagrammodel{T}{Y}{fall}

% Render the spring model:
\switchmodelprefix{spring}\textmodel\computemodel\diagrammodel{T}{X}{spring}
```

Each prefix carries an independent rule list, stop condition, and recorded series.

8 Requirements

`numodel` requires LuaLaTeX (the engine, for the Lua runtime) and TeX Live 2022 or later. Mandatory dependencies: `expl3`, `xparse`, `l3keys2e`, `amsmath`, `amssymb`, `tikz`, `luacode`, `siunitx`, `float`, and the sibling package `numodel-plot` (which itself pulls in `pgfplots`). The companion Lua module `numodel.lua` must be installed alongside the `.sty` in a directory searched by `kpse`.

9 Implementation

```
1 (*package)
2 \NeedsTeXFormat{LaTeX2e}
3 \ProvidesPackage{numodel}[2026/05/17 v0.3.0
4 Numerical physics models with Euler integration and Forrester diagrams]
5
6 \RequirePackage{expl3}
7 \RequirePackage{xparse}
8 \RequirePackage{l3keys2e}
9 \RequirePackage{amsmath}
10 \RequirePackage{amssymb} % \leqslant / \geqslant in rendered <= / >=
11 \RequirePackage{tikz}
12 \usetikzlibrary{shapes.symbols}
13 \usetikzlibrary{arrows.meta}
```

```

14 % Custom Cloud arrow tip used as the open end of inflow / outflow
15 % pipes. The silhouette consists of nine concatenated elliptical
16 % arcs (\pgfpatharc): each arc starts at the current path point,
17 % which pgf treats as lying on an ellipse at angle <start>, and
18 % sweeps through to angle <end>. Varying the start and end angles
19 % per arc produces zigzagging half-circle puffs -- the
20 % characteristic cloud outline. A single factor \puffscale sets the
21 % rx and ry of every ellipse and thus the size of all puffs at once.
22 % The chain has a horizontal span of  $(4+2\sqrt{2})\text{rx} \sim 6.83\text{rx}$ ;
23 % with puffscale=0.22 that is  $\sim 1.5*\text{pgfarrowlength}$ . The path
24 % therefore starts at  $-0.5*\text{pgfarrowlength}$  so the silhouette covers
25 %  $[-0.5L, L]$  in local arrow coordinates and the apex lands exactly
26 % at  $\text{pgfarrowlength}$  -- where the built-in tips (Latex, Stealth,
27 % ...) also place their tip. The line width is set explicitly to
28 % 0.3pt, independent of the arrow's line width, so a thick arrow
29 % does not also get a thick cloud outline.
30 \pgfdeclarearrow{
31   name = Cloud,
32   parameters = { \the\pgfarrowlength \the\pgfarrowwidth },
33   setup code = {
34     \pgfarrowssettipend{\pgfarrowlength}
35     \pgfarrowssetbackend{-0.5\pgfarrowlength}
36     \pgfarrowssetlineend{-0.5\pgfarrowlength}
37     \pgfarrowssetvisualbackend{-0.5\pgfarrowlength}
38     \pgfarrowssetvisualtipend{\pgfarrowlength}
39   },
40   defaults = { length = 1em, width = 1em },
41   drawing code = {
42     \def\puffscale{0.22}
43     \edef\puffrx{\puffscale\pgfarrowlength}
44     \edef\puffry{\puffscale\pgfarrowwidth}
45     \pgfsetlinewidth{0.3pt}
46     \pgfpathmoveto{\pgfqpoint{-0.5\pgfarrowlength}{0pt}}
47     \pgfpatharc{180}{ 90}{\puffrx\space and \puffry}
48     \pgfpatharc{225}{ 45}{\puffrx\space and \puffry}
49     \pgfpatharc{180}{ 0}{\puffrx\space and \puffry}
50     \pgfpatharc{135}{-45}{\puffrx\space and \puffry}
51     \pgfpatharc{ 90}{-90}{\puffrx\space and \puffry}
52     \pgfpatharc{ 45}{-135}{\puffrx\space and \puffry}
53     \pgfpatharc{ 0}{-180}{\puffrx\space and \puffry}
54     \pgfpatharc{-45}{-225}{\puffrx\space and \puffry}
55     \pgfpatharc{-90}{-180}{\puffrx\space and \puffry}
56     \pgfpathclose
57     \pgfusepathqfillstroke
58   },
59 }
60 \RequirePackage{luacode}
61 \RequirePackage{siunitx}
62 \RequirePackage{float} % \diagrammodel uses [H] placement
63 \RequirePackage{nummodel-plot}
64
65 % =====
66 % Non-expl3 helpers: ` \nm@def@num@cs` / ` \nm@def@qty@cs` / ` \nm@def@pre@cs`
67 % define ` \<fullname><suffix>` macros whose body contains literal
68 % siunitx options. Defined OUTSIDE \ExplSyntaxOn so the option-string
69 % colons (`-3:n`, `-0:0`) and other punctuation keep normal catcodes.
70 % Storing those bodies inside an expl3-context would tokenise `:` as
71 % letter, which then breaks siunitx's option parser at expansion time.
72 % =====
73 \newcommand{\NummodelDefNumCs}[3]{% #1 = fullname, #2 = start expr, #3 = sigfigs
74   \expandafter\gdef\csname #1num\endcsname{%
75     \num[evaluate-expression=true,
76       round-mode=figures, round-precision=#3,
77       exponent-mode=threshold, exponent-thresholds=-3:#3]
78     {#2}\relax
79   }%

```

```

80 }
81 \newcommand{\NumodelDefQtyCs}[4]{% #1 = fullname, #2 = expr, #3 = sigfigs, #4 = unit
82   \expandafter\gdef\csname #1qty\endcsname{%
83     \qty[evaluate-expression=true,
84       round-mode=figures, round-precision=#3,
85       exponent-mode=threshold, exponent-thresholds=-3:#3]
86     {#2}{#4}\relax
87   }%
88 }
89 \newcommand{\NumodelDefPreCs}[4]{% same parameters as above
90   \expandafter\gdef\csname #1pre\endcsname{%
91     \qty[evaluate-expression=true,
92       round-mode=figures, round-precision=#3,
93       prefix-mode=combine-exponent,
94       exponent-mode=engineering,
95       exponent-thresholds=-0:0]
96     {#2}{#4}\relax
97   }%
98 }
99
100 \ExplSyntaxOn
101
102 % =====
103 % Lua module for data storage (O(1) append, O(N) total)
104 % =====
105 % All variable values are stored per step in Lua tables. This
106 % replaces the O(N^2) \xdef accumulation for coordinates and makes
107 % min/max tracking efficient. After \computemodel the results are
108 % pushed back to TeX macros.
109
110 \begin{luacode*}
111 local f = kpse.find_file("numodel.lua", "tex")
112 if f and f ~= "" then
113   dofile(f)
114 else
115   tex.error("numodel: cannot find companion Lua module numodel.lua."
116     .. " Install it in a directory searched by kpse"
117     .. " (e.g. TEXMFHOME/tex/lualatex/numodel/).")
118 end
119 \end{luacode*}
120
121 % =====
122 % Internal data structures
123 % =====
124
125 \seq_new:N \g_mvar_names_seq      % all model variable names
126 \seq_new:N \g_mvar_start_seq      % names with an initial value (for the table)
127 \seq_new:N \g_mrrule_seq          % model rules (display strings)
128 \seq_new:N \g_mrrule_type_seq     % type per display row: rule | cont
129 \seq_new:N \g_mrrule_calc_seq     % model rules for execution: {name}{expr}
130 \int_new:N \g_mrrule_counter_int  % rule counter (display numbering)
131 \tl_new:N \g_numodel_stop_expr_tl % stop-condition expression for execution
132 \int_new:N \g_numodel_steps_int   % number of executed steps
133 \int_new:N \g_numodel_maxiter_int % safety limit (init via \numodelsetup)
134 \int_new:N \g_numodel_gridmaxx_int % \graphicmodel wrap threshold (0 = off)
135
136 % --- Graphic-model infrastructure (Phase 1) ---
137 \tl_new:N \l__numodel_tmp_tl      % temporary helper
138 \tl_new:N \l__numodel_scratch_tl  % scratch for type/text checks
139 \tl_new:N \l__numodel_scratch_y_tl % scratch y-coord (svg lookup)
140
141 % Diagram style for \graphicmodel (see \numodelsetup):
142 %   tight      -- valve takes the label of the direct inflow variable;
143 %               the aux/const is moved to the valve position and not
144 %               drawn as a separate node (compact, default).
145 %   forrester -- valve has no label; the aux/const stays at its own

```

```

146 %          gridy position; causal arrow from aux/const to the
147 %          valve.
148 %   edu      -- combination: valve takes the label *and* a separate
149 %          aux/const node remains with a causal arrow to it.
150 \tl_new:N \g__numodel_diagram_style_tl
151
152 % Sub-keys that fine-tune the appearance of the Forrester diagram
153 % (see \numodelsetup). Empty value = "follow diagram-style default".
154 %   flowarrow-style      : hollow | filled
155 %          forrester -> hollow, tight|edu -> filled
156 %   valve-style         : valve | circle | edu
157 %          forrester -> valve, tight|edu -> edu
158 %   flowarrow-cloud-tip: true | false
159 %          forrester -> true, tight|edu -> false
160 % Per-variable override for flowarrow-cloud-tip is stored in
161 % \<name>flowcloud (set via the [flowarrow-cloud-tip=...] key on
162 % \mvar; empty means "follow global key").
163 \tl_new:N \g__numodel_flowarrow_style_tl
164 \tl_new:N \g__numodel_valve_style_tl
165 \tl_new:N \g__numodel_flowcloud_tl
166
167 % Units in the \textmodel initial-values column (see \numodelsetup):
168 %   true  (default) -- initial value rendered via \<name>qty
169 %          (number + unit)
170 %   false          -- initial value rendered via \<name>num
171 %          (number only)
172 % Per-table override via \textmodel[units=true|false].
173 \bool_new:N \g__numodel_units_bool
174
175 % Decimal separator for \textmodel initial values and \diagrammodel
176 % tick values (see \numodelsetup{decimal-separator}):
177 %   point -- '.' (siunitx output-decimal-marker={.})
178 %   comma -- ',' (siunitx output-decimal-marker={,})
179 % The default follows syntax: english -> point, coachtaal -> comma.
180 % An explicit decimal-separator key sets
181 % \g__numodel_dsep_explicit_bool so that a later syntax change no
182 % longer overrules the choice.
183 \tl_new:N \g__numodel_dsep_tl
184 \bool_new:N \g__numodel_dsep_explicit_bool
185
186 % =====
187 % Syntax lookup
188 % =====
189 % Determines the language of the text-rendered model. Affects display
190 % only (\textmodel, \mruletext, \mstop); \computemodel always uses
191 % \fpeval internally and is language-agnostic. Each value of
192 % \g__numodel_syntax_tl is a tag that selects both the backing
193 % \_numodel_kw_<tag>_<key>: macros and the file numodel-<tag>.def
194 % that defines them; the package ships EN (English/XMILE) and NL
195 % (Dutch/CoachTaal).
196 %
197 % The CTAN default is EN. The initial value is set below via
198 % \numodelsetup.
199
200 \tl_new:N \g__numodel_syntax_tl
201
202 % Lookup (fully expandable): \_numodel_kw:n {<key>} returns the
203 % translation in the current syntax language. The key also controls
204 % the surrounding spaces (see keys 'then' vs 'then_nl' etc.). The
205 % backing macros \_numodel_kw_<LANG>_<key>: are provided by the
206 % language file numodel-<LANG>.def loaded by \_numodel_load_syntax_def:n
207 % (see below).
208 \cs_new:Npn \_numodel_kw:n #1
209 { \use:c { \_numodel_kw_ \g__numodel_syntax_tl _ #1 : } }
210
211 % Pre-computed translations in tl variables so they are usable via

```

```

212 % \u{} in l3regex replacement text. (l3regex treats {...} in the
213 % replacement as brace groups, so a bare \_numodel_kw:n{key} call
214 % fails there.)
215 \tl_new:N \g__numodel_kw_if_tl
216 \tl_new:N \g__numodel_kw_then_tl
217 \tl_new:N \g__numodel_kw_then_nl_tl
218 \tl_new:N \g__numodel_kw_else_tl
219 \tl_new:N \g__numodel_kw_else_nl_tl
220 \tl_new:N \g__numodel_kw_endif_tl
221 \tl_new:N \g__numodel_kw_endif_nl_tl
222 \tl_new:N \g__numodel_kw_and_tl
223 \tl_new:N \g__numodel_kw_or_tl
224 \tl_new:N \g__numodel_kw_not_tl
225 \tl_new:N \g__numodel_kw_sign_tl
226 \tl_new:N \g__numodel_kw_abs_tl
227 \tl_new:N \g__numodel_kw_sqrt_tl
228 \tl_new:N \g__numodel_kw_exp_tl
229 \tl_new:N \g__numodel_kw_ln_tl
230 \tl_new:N \g__numodel_kw_sin_tl
231 \tl_new:N \g__numodel_kw_cos_tl
232 \tl_new:N \g__numodel_kw_tan_tl
233 \tl_new:N \g__numodel_kw_asin_tl
234 \tl_new:N \g__numodel_kw_acos_tl
235 \tl_new:N \g__numodel_kw_stop_tl
236
237 % Recomputes all tl caches from \g__numodel_syntax_tl. Must be
238 % called after any change to the syntax language.
239 \cs_new_protected:Npn \_numodel_refresh_kw:
240 {
241   \tl_gset:Nx \g__numodel_kw_if_tl      { \_numodel_kw:n {if}      }
242   \tl_gset:Nx \g__numodel_kw_then_tl    { \_numodel_kw:n {then}    }
243   \tl_gset:Nx \g__numodel_kw_then_nl_tl { \_numodel_kw:n {then_nl}  }
244   \tl_gset:Nx \g__numodel_kw_else_tl    { \_numodel_kw:n {else}    }
245   \tl_gset:Nx \g__numodel_kw_else_nl_tl { \_numodel_kw:n {else_nl}  }
246   \tl_gset:Nx \g__numodel_kw_endif_tl   { \_numodel_kw:n {endif}   }
247   \tl_gset:Nx \g__numodel_kw_endif_nl_tl { \_numodel_kw:n {endif_nl} }
248   \tl_gset:Nx \g__numodel_kw_and_tl     { \_numodel_kw:n {and}     }
249   \tl_gset:Nx \g__numodel_kw_or_tl      { \_numodel_kw:n {or}      }
250   \tl_gset:Nx \g__numodel_kw_not_tl     { \_numodel_kw:n {not}     }
251   \tl_gset:Nx \g__numodel_kw_sign_tl    { \_numodel_kw:n {sign}    }
252   \tl_gset:Nx \g__numodel_kw_abs_tl     { \_numodel_kw:n {abs}     }
253   \tl_gset:Nx \g__numodel_kw_sqrt_tl    { \_numodel_kw:n {sqrt}    }
254   \tl_gset:Nx \g__numodel_kw_exp_tl     { \_numodel_kw:n {exp}     }
255   \tl_gset:Nx \g__numodel_kw_ln_tl     { \_numodel_kw:n {ln}     }
256   \tl_gset:Nx \g__numodel_kw_sin_tl    { \_numodel_kw:n {sin}     }
257   \tl_gset:Nx \g__numodel_kw_cos_tl    { \_numodel_kw:n {cos}     }
258   \tl_gset:Nx \g__numodel_kw_tan_tl    { \_numodel_kw:n {tan}     }
259   \tl_gset:Nx \g__numodel_kw_asin_tl    { \_numodel_kw:n {asin}    }
260   \tl_gset:Nx \g__numodel_kw_acos_tl    { \_numodel_kw:n {acos}    }
261   \tl_gset:Nx \g__numodel_kw_stop_tl    { \_numodel_kw:n {stop}    }
262 }
263
264 % Helper: \_numodel_kwt:n {<key>} expands to \text{<kw-value>}.
265 % Meant for use inside \tl_set:Nx / \seq_gput_right:Nx -- the kw is
266 % inserted during e-expansion while \text remains protected.
267 \cs_new:Npn \_numodel_kwt:n #1 { \text { \_numodel_kw:n {#1} } }
268
269 % Language-file loader. Each tag <LANG> is backed by a file
270 % numodel-<LANG>.def installed in a kpse-searched directory; the
271 % package ships numodel-EN.def and numodel-NL.def, users can drop
272 % additional files in TEXMFHOME/tex/latex/numodel/ without rebuilding
273 % the package.
274 %
275 % A small alias property maps the historical long names to the
276 % canonical two-letter tag used in the file name and the internal
277 % macro names. Users who add their own file just pass the file's tag

```

```

278 % to syntax=<tag> directly; no alias is required.
279 \prop_new:N \g__numodel_syntax_aliases_prop
280 \prop_gput:Nnn \g__numodel_syntax_aliases_prop { english } { EN }
281 \prop_gput:Nnn \g__numodel_syntax_aliases_prop { coachtaal } { NL }
282 \prop_gput:Nnn \g__numodel_syntax_aliases_prop { dutch } { NL }
283
284 % Tags whose .def file has already been input during this run, so the
285 % lookup only triggers \InputIfFileExists the first time.
286 \seq_new:N \g__numodel_loaded_syntax_seq
287
288 \msg_new:nnn { numodel } { unknown-syntax }
289 {
290   Cannot-load-syntax~'#1':~file~'numodel-#1.def'~not~found~by~
291   kpse.~The~package~ships~with~EN~(English)~and~NL~(Dutch~
292   CoachTaal);~drop~your~own~numodel-<LANG>.def~in~
293   TEXMFHOME/tex/latex/numodel/~to~add~more~languages.
294 }
295
296 % \__numodel_load_syntax_def:n {<tag>}
297 % Inputs numodel-<tag>.def the first time it is requested. Raises a
298 % LaTeX error if the file is not found by kpse.
299 \cs_new_protected:Npn \__numodel_load_syntax_def:n #1
300 {
301   \seq_if_in:NnF \g__numodel_loaded_syntax_seq {#1}
302   {
303     \InputIfFileExists { numodel-#1.def }
304     { \seq_gput_right:Nn \g__numodel_loaded_syntax_seq {#1} }
305     { \msg_error:nnn { numodel } { unknown-syntax } {#1} }
306   }
307 }
308
309 \tl_new:N \l__numodel_syntax_tag_tl
310
311 % \__numodel_set_syntax:n {<value>}
312 % Public-facing setter behind the syntax key. Resolves aliases,
313 % loads the matching .def file (once), publishes the canonical tag
314 % in \g__numodel_syntax_tl, refreshes the keyword cache, and -- when
315 % the user has not pinned a decimal separator explicitly -- adopts the
316 % language's preferred decimal mark via the optional
317 % \__numodel_kw_<LANG>_dsep_default: hook.
318 \cs_new_protected:Npn \__numodel_set_syntax:n #1
319 {
320   \prop_get:NnNF \g__numodel_syntax_aliases_prop {#1}
321   \l__numodel_syntax_tag_tl
322   { \tl_set:Nn \l__numodel_syntax_tag_tl {#1} }
323   \exp_args:NV \__numodel_load_syntax_def:n \l__numodel_syntax_tag_tl
324   \tl_gset_eq:NN \g__numodel_syntax_tl \l__numodel_syntax_tag_tl
325   \__numodel_refresh_kw:
326   \bool_if:NF \g__numodel_dsep_explicit_bool
327   {
328     \cs_if_exist:cTF
329     { __numodel_kw_ \g__numodel_syntax_tl _dsep_default: }
330     { \tl_gset:Ne \g__numodel_dsep_tl { \__numodel_kw:n {dsep_default} } }
331     { \tl_gset:Nn \g__numodel_dsep_tl { point } }
332   }
333 }
334
335 % Applies the decimal separator (only for the duration of the
336 % surrounding TeX group, so a document-wide \sisetup is left
337 % untouched). Calls both siunitx (for \num/\qty in \textmodel
338 % initial values) and pgfplots' tick styles (for \diagrammodel tick
339 % labels) so the choice is consistent everywhere.
340 \cs_new_protected:Npn \__numodel_apply_dsep:
341 {
342   \str_if_eq:VnTF \g__numodel_dsep_tl { comma }
343   {

```

```

344 \sisetup{ output-decimal-marker = {,} }
345 % Append behind the existing numodel/axis style so our
346 % xticklabel-style replaces what numodel-plot hard-codes.
347 % The \pgfplotsset mutation is \def-based and hence
348 % group-local.
349 \pgfplotsset
350 {
351     numodel/axis/.append-style=
352     {
353         xticklabel-style=
354         { /pgf/number-format/.cd, use-comma, /tikz/.cd } ,
355         yticklabel-style=
356         { /pgf/number-format/.cd, use-comma, /tikz/.cd } ,
357     }
358 }
359 {
360 \sisetup{ output-decimal-marker = {.} }
361 \pgfplotsset
362 {
363     numodel/axis/.append-style=
364     {
365         xticklabel-style=
366         { /pgf/number-format/.cd, use-period, /tikz/.cd } ,
367         yticklabel-style=
368         { /pgf/number-format/.cd, use-period, /tikz/.cd } ,
369     }
370 }
371 }
372 }
373 }
374
375 % =====
376 % Configuration command \numodelsetup
377 % =====
378 % Runtime API for settings. Keys:
379 % syntax      -- language tag (file numodel-<tag>.def must be on kpse
380 %              path). Built-in: EN, NL. Legacy aliases: english,
381 %              coachtaal, dutch.
382 % maxiter     -- safety limit for \computemodel (default 20000)
383 % graphscalex -- horizontal grid spacing \dgridx for Forrester
384 %              diagrams (default 2)
385 % graphscaley -- vertical grid spacing \dgridy for Forrester
386 %              diagrams (default 2)
387 % stockwidth  -- half-width of the stock rectangle (default 0.375)
388
389 % Helper for choice-with-empty-reset: validates #4 against #3 (a
390 % comma-separated whitelist) and writes it into #1 (a tl). An empty
391 % value clears the tl (which means "follow the diagram-style
392 % default" for the flowarrow-style / valve-style /
393 % flowarrow-cloud-tip keys). Any other value triggers a warning.
394 \msg_new:nnn { numodel } { bad-choice }
395 { Key~'#1'~accepts-only~#2,~or~empty~to~reset.~Got:~'#3'. }
396
397 \cs_new_protected:Npn \__numodel_setup_choice:Nnnn #1 #2 #3 #4
398 {
399     \tl_if_blank:nTF {#4}
400     { \tl_gclear:N #1 }
401     {
402         \clist_if_in:nnTF {#3} {#4}
403         { \tl_gset:Nn #1 {#4} }
404         { \msg_warning:nnnnn { numodel } { bad-choice } {#2} {#3} {#4} }
405     }
406 }
407
408 % Local-tl variant: same validation, local set/clear. Used by the
409 % \graphicmodel one-render override and the per-\mvar override.

```



```

410 \cs_new_protected:Npn \__numodel_local_choice:Nnnn #1 #2 #3 #4
411 {
412   \tl_if_blank:nTF {#4}
413   { \tl_clear:N #1 }
414   {
415     \clist_if_in:nnTF {#3} {#4}
416     { \tl_set:Nn #1 {#4} }
417     { \msg_warning:nnnnn { numodel } { bad-choice } {#2} {#3} {#4} }
418   }
419 }
420
421 \keys_define:nn { numodel / setup }
422 {
423   syntax .code:n =
424   { \__numodel_set_syntax:n {#1} },
425   maxiter .int_gset:N = \g_numodel_maxiter_int,
426   graphscalex .code:n = { \tl_gset:Nn \dgridx {#1} },
427   graphscaley .code:n = { \tl_gset:Nn \dgridy {#1} },
428   stockwidth .code:n = { \tl_gset:Nn \halfstockwidth {#1} },
429   gridmaxx .int_gset:N = \g_numodel_gridmaxx_int,
430   diagram-style .choice:,
431   diagram-style / tight .code:n =
432   { \tl_gset:Nn \g_numodel_diagram_style_tl { tight } },
433   diagram-style / forrester .code:n =
434   { \tl_gset:Nn \g_numodel_diagram_style_tl { forrester } },
435   diagram-style / edu .code:n =
436   { \tl_gset:Nn \g_numodel_diagram_style_tl { edu } },
437   flowarrow-style .code:n =
438   { \__numodel_setup_choice:Nnnn \g_numodel_flowarrow_style_tl
439     { flowarrow-style } { hollow , filled } {#1} },
440   valve-style .code:n =
441   { \__numodel_setup_choice:Nnnn \g_numodel_valve_style_tl
442     { valve-style } { valve , circle , edu } {#1} },
443   flowarrow-cloud-tip .code:n =
444   { \__numodel_setup_choice:Nnnn \g_numodel_flowcloud_tl
445     { flowarrow-cloud-tip } { true , false } {#1} },
446   units .choice:,
447   units / true .code:n =
448   { \bool_gset_true:N \g_numodel_units_bool },
449   units / false .code:n =
450   { \bool_gset_false:N \g_numodel_units_bool },
451   decimal-separator .choice:,
452   decimal-separator / comma .code:n =
453   {
454     \tl_gset:Nn \g_numodel_dsep_tl { comma }
455     \bool_gset_true:N \g_numodel_dsep_explicit_bool
456   },
457   decimal-separator / point .code:n =
458   {
459     \tl_gset:Nn \g_numodel_dsep_tl { point }
460     \bool_gset_true:N \g_numodel_dsep_explicit_bool
461   },
462 }
463
464 \NewDocumentCommand{\numodelsetup}{m}
465 { \keys_set:nn { numodel / setup } {#1} }
466
467 % Defaults (this call also initialises \g_numodel_syntax_tl,
468 % \g_numodel_maxiter_int, \dgridx, \dgridy, \halfstockwidth and
469 % \g_numodel_diagram_style_tl).
470 \numodelsetup
471 {
472   syntax = EN,
473   maxiter = 20000,
474   graphscalex = 2,
475   graphscaley = 2,

```

```

476     stockwidth      = 0.375,
477     diagram-style = tight,
478     units           = true,
479 }
480
481 % Package-time options. \usepackage[syntax=NL]{numodel} delegates
482 % to the same key infrastructure as \numodelsetup.
483 \keys_define:nn { numodel / pkg }
484 {
485     syntax          .meta:nn = { numodel / setup }{ syntax          = #1 },
486     maxiter         .meta:nn = { numodel / setup }{ maxiter         = #1 },
487     graphscalex     .meta:nn = { numodel / setup }{ graphscalex     = #1 },
488     graphscaley     .meta:nn = { numodel / setup }{ graphscaley     = #1 },
489     stockwidth      .meta:nn = { numodel / setup }{ stockwidth      = #1 },
490     gridmaxx        .meta:nn = { numodel / setup }{ gridmaxx        = #1 },
491     diagram-style   .meta:nn = { numodel / setup }{ diagram-style   = #1 },
492     flowarrow-style .meta:nn = { numodel / setup }{ flowarrow-style = #1 },
493     valve-style     .meta:nn = { numodel / setup }{ valve-style     = #1 },
494     flowarrow-cloud-tip .meta:nn = { numodel / setup }{ flowarrow-cloud-tip = #1 },
495     units           .meta:nn = { numodel / setup }{ units           = #1 },
496     decimal-separator .meta:nn = { numodel / setup }{ decimal-separator = #1 },
497 }
498 \ProcessKeyOptions [ numodel / pkg ]
499
500 % =====
501 % Prefix system
502 % =====
503 % Each model lives under a prefix (a short string). The "live state"
504 % sits in the \g_mvar_*, \g_mrul_*, \g_numodel_* variables above;
505 % \newmodelprefix and \switchmodelprefix swap that state to/from
506 % per-prefix backup storage.
507 %
508 % - \g_numodel_current_prefix_tl: current prefix (empty at package load)
509 % - \g_numodel_prefixes_seq: list of all registered prefixes
510 % - \l_numodel_cmd_prefix_tl: prefix passed via [prefix=...] key
511 % - \l_numodel_eff_prefix_tl: effective prefix for the current command
512
513 \tl_new:N \g_numodel_current_prefix_tl
514 \seq_new:N \g_numodel_prefixes_seq
515 \tl_new:N \l_numodel_cmd_prefix_tl
516 \tl_new:N \l_numodel_eff_prefix_tl
517 \tl_new:N \l_numodel_fullname_tl
518
519 % Per-prefix storage: on swap the current state is copied to
520 % \g_numodel_<P>_<slot>_{seq,tl,int,prop}. Load goes the other way.
521 \cs_new_protected:Npn \__numodel_save_state:n #1
522 {
523     \seq_gset_eq:cN { g__numodel_ #1 _vars_seq      } \g_mvar_names_seq
524     \seq_gset_eq:cN { g__numodel_ #1 _starts_seq     } \g_mvar_start_seq
525     \seq_gset_eq:cN { g__numodel_ #1 _rules_seq      } \g_mrul_seq
526     \seq_gset_eq:cN { g__numodel_ #1 _ruletypes_seq  } \g_mrul_type_seq
527     \seq_gset_eq:cN { g__numodel_ #1 _rulecalc_seq   } \g_mrul_calc_seq
528     \int_gset:cn { g__numodel_ #1 _rulecounter_int }
529     { \int_use:N \g_mrul_counter_int }
530     \tl_gset_eq:cN { g__numodel_ #1 _stopexpr_tl    } \g_numodel_stop_expr_tl
531     \int_gset:cn { g__numodel_ #1 _steps_int        }
532     { \int_use:N \g_numodel_steps_int }
533 }
534
535 \cs_new_protected:Npn \__numodel_load_state:n #1
536 {
537     \seq_gset_eq:Nc \g_mvar_names_seq { g__numodel_ #1 _vars_seq      }
538     \seq_gset_eq:Nc \g_mvar_start_seq { g__numodel_ #1 _starts_seq     }
539     \seq_gset_eq:Nc \g_mrul_seq       { g__numodel_ #1 _rules_seq      }
540     \seq_gset_eq:Nc \g_mrul_type_seq  { g__numodel_ #1 _ruletypes_seq  }
541     \seq_gset_eq:Nc \g_mrul_calc_seq  { g__numodel_ #1 _rulecalc_seq   }

```

```

542     \int_gset:Nn \g_mrule_counter_int
543     { \int_use:c { g__numodel_ #1 _rulecounter_int } }
544     \tl_gset_eq:Nc \g_numodel_stop_expr_tl { g__numodel_ #1 _stopexpr_tl }
545     \int_gset:Nn \g_numodel_steps_int
546     { \int_use:c { g__numodel_ #1 _steps_int } }
547 }
548
549 \cs_new_protected:Npn \__numodel_clear_state:
550 {
551     \seq_gclear:N \g_mvar_names_seq
552     \seq_gclear:N \g_mvar_start_seq
553     \seq_gclear:N \g_mrule_seq
554     \seq_gclear:N \g_mrule_type_seq
555     \seq_gclear:N \g_mrule_calc_seq
556     \int_gzero:N \g_mrule_counter_int
557     \tl_gclear:N \g_numodel_stop_expr_tl
558     \int_gzero:N \g_numodel_steps_int
559 }
560
561 % Initialise per-prefix backup variables (once, at \newmodelprefix).
562 \cs_new_protected:Npn \__numodel_init_backup:n #1
563 {
564     \seq_new:c { g__numodel_ #1 _vars_seq }
565     \seq_new:c { g__numodel_ #1 _starts_seq }
566     \seq_new:c { g__numodel_ #1 _rules_seq }
567     \seq_new:c { g__numodel_ #1 _ruletypes_seq }
568     \seq_new:c { g__numodel_ #1 _rulecalc_seq }
569     \int_new:c { g__numodel_ #1 _rulecounter_int }
570     \tl_new:c { g__numodel_ #1 _stopexpr_tl }
571     \int_new:c { g__numodel_ #1 _steps_int }
572 }
573
574 % Public commands for prefix management
575 \NewDocumentCommand{\newmodelprefix}{ m }
576 {
577     \typeout{NEWPREFIX~start:~#1}
578     \seq_if_in:NnTF \g_numodel_prefixes_seq {#1}
579     { \msg_error:nnn { numodel } { prefix-exists } {#1} }
580     {
581         \typeout{NEWPREFIX~save~current:~\g_numodel_current_prefix_tl}
582         % Save current state under the previous prefix (if any)
583         \tl_if_empty:NF \g_numodel_current_prefix_tl
584         { \exp_args:NV \__numodel_save_state:n \g_numodel_current_prefix_tl }
585         \typeout{NEWPREFIX~register}
586         % Register new prefix + init backup vars + Lua state
587         \seq_gput_right:Nn \g_numodel_prefixes_seq {#1}
588         \typeout{NEWPREFIX~init~backup}
589         \__numodel_init_backup:n {#1}
590         \typeout{NEWPREFIX~lua~init}
591         \directlua{ numodel.init_prefix("#1") }
592         \typeout{NEWPREFIX~clear~state}
593         % Clear current state and set the prefix
594         \__numodel_clear_state:
595         \typeout{NEWPREFIX~set~current}
596         \tl_gset:Nn \g_numodel_current_prefix_tl {#1}
597         \typeout{NEWPREFIX~done:~#1}
598     }
599 }
600
601 \NewDocumentCommand{\switchmodelprefix}{ m }
602 {
603     \seq_if_in:NnTF \g_numodel_prefixes_seq {#1}
604     {
605         % Save current state (if any), load the new one
606         \tl_if_empty:NF \g_numodel_current_prefix_tl
607         { \exp_args:NV \__numodel_save_state:n \g_numodel_current_prefix_tl }

```

```

608     \__numodel_load_state:n {#1}
609     \tl_gset:Nn \g_numodel_current_prefix_tl {#1}
610 }
611 { \msg_error:nnn { numodel } { prefix-unknown } {#1} }
612 }
613
614 % Iterate over all variables from all prefixes.
615 % #1 = code that uses ##1 = full variable name (prefix + short).
616 % Public registration API for external tools such as worksheet.tex.
617 \NewDocumentCommand{\NumodelForEachVar}{+m }
618 {
619     \seq_map_inline:Nn \g_numodel_prefixes_seq
620     {
621         \seq_map_inline:cn { g__numodel_ ##1 _vars_seq } {#1}
622     }
623 }
624
625 % Resolve the effective prefix for a command call. Takes
626 % [prefix=<p>] from the keyval argument; otherwise falls back to the
627 % current prefix. Stores the result in \l__numodel_eff_prefix_tl.
628 \keys_define:nn { numodel / cmd }
629 {
630     prefix .tl_set:N = \l__numodel_cmd_prefix_tl,
631     % \graphicmodel-only: temporary override of the global
632     % diagram-style. Cleared after every \graphicmodel call.
633     diagram-style .choice:,
634     diagram-style / tight .code:n =
635     { \tl_set:Nn \l__numodel_cmd_diagstyle_tl { tight } },
636     diagram-style / forrester .code:n =
637     { \tl_set:Nn \l__numodel_cmd_diagstyle_tl { forrester } },
638     diagram-style / edu .code:n =
639     { \tl_set:Nn \l__numodel_cmd_diagstyle_tl { edu } },
640     % \graphicmodel-only: temporary overrides of the corresponding
641     % global flowarrow/valve/cloud keys. Empty after every call.
642     flowarrow-style .code:n =
643     { \__numodel_local_choice:Nnnn \l__numodel_cmd_flowarrow_tl
644       { flowarrow-style } { hollow , filled } {#1} },
645     valve-style .code:n =
646     { \__numodel_local_choice:Nnnn \l__numodel_cmd_valve_tl
647       { valve-style } { valve , circle , edu } {#1} },
648     flowarrow-cloud-tip .code:n =
649     { \__numodel_local_choice:Nnnn \l__numodel_cmd_flowcloud_tl
650       { flowarrow-cloud-tip } { true , false } {#1} },
651     % \textmodel-only: temporary override of the global units bool.
652     % Cleared after every \textmodel call.
653     units .choice:,
654     units / true .code:n =
655     { \tl_set:Nn \l__numodel_cmd_units_tl { true } },
656     units / false .code:n =
657     { \tl_set:Nn \l__numodel_cmd_units_tl { false } },
658 }
659 \tl_new:N \l__numodel_cmd_diagstyle_tl
660 \tl_new:N \l__numodel_cmd_flowarrow_tl
661 \tl_new:N \l__numodel_cmd_valve_tl
662 \tl_new:N \l__numodel_cmd_flowcloud_tl
663 \tl_new:N \l__numodel_cmd_units_tl
664
665 \cs_new_protected:Npn \__numodel_resolve_prefix:n #1
666 {
667     \tl_clear:N \l__numodel_cmd_prefix_tl
668     \keys_set:nn { numodel / cmd } {#1}
669     \tl_if_empty:NTF \l__numodel_cmd_prefix_tl
670     { \tl_set_eq:NN \l__numodel_eff_prefix_tl \g_numodel_current_prefix_tl }
671     { \tl_set_eq:NN \l__numodel_eff_prefix_tl \l__numodel_cmd_prefix_tl }
672 }
673

```

```

674 % Execute code under a temporarily different prefix (via state swap).
675 % #1 = target prefix (tl, in variable)
676 % #2 = code
677 \cs_new_protected:Npn \__numodel_with_prefix:Nn #1 #2
678 {
679     \tl_if_eq:NNTF #1 \g_numodel_current_prefix_tl
680     { #2 } % already the current prefix; no swap needed
681     {
682         \tl_set_eq:NN \l__numodel_saved_prefix_tl \g_numodel_current_prefix_tl
683         \exp_args:NV \switchmodelprefix #1
684         #2
685         \exp_args:NV \switchmodelprefix \l__numodel_saved_prefix_tl
686     }
687 }
688 \tl_new:N \l__numodel_saved_prefix_tl
689
690 % Build the full name = <prefix><shortname>. Stored in
691 % \l__numodel_fullname_tl.
692 \cs_new_protected:Npn \__numodel_set_fullname:n #1
693 {
694     \tl_set:Ne \l__numodel_fullname_tl { \l__numodel_eff_prefix_tl #1 }
695 }
696
697 % Keys for the optional argument of \mvar (grid position +
698 % initial-value aliases + prefix).
699 \tl_new:N \l__numodel_alias_tl
700 \tl_new:N \l__numodel_aliasleft_tl
701 \tl_new:N \l__numodel_aliasright_tl
702
703 \keys_define:nn { numodel / mvar }
704 {
705     prefix      .tl_set:N = \l__numodel_cmd_prefix_tl ,
706     gridx       .tl_set:N = \l__numodel_gridx_tl ,
707     gridy       .tl_set:N = \l__numodel_gridy_tl ,
708     alias       .tl_set:N = \l__numodel_alias_tl ,
709     aliasleft   .tl_set:N = \l__numodel_aliasleft_tl ,
710     aliasright  .tl_set:N = \l__numodel_aliasright_tl ,
711     flowarrow-cloud-tip .code:n =
712     { \__numodel_local_choice:Nnnn \l__numodel_mvar_flowcloud_tl
713       { flowarrow-cloud-tip } { true , false } {#1} },
714 }
715 \tl_new:N \l__numodel_mvar_flowcloud_tl
716
717 % =====
718 % \mvar[keys]{name}{display}{startvalue}{unit}{sig}{type}
719 % =====
720 % Declare a model variable. Generates the following macros:
721 %
722 % \<name>      -- numeric value (\edef + \fpeval), or warning if empty
723 % \<name>text  -- display name for the model table (e.g. F_{res})
724 % \<name>unit  -- SI unit (e.g. kN)
725 % \<name>unitraw - raw SI unit (e.g. \kilo\N)
726 % \<name>sign  -- number of significant figures
727 % \<name>type  -- variable type: stock, constant, aux, system
728 %                (Dutch synonyms voorraad/constante/hulp/systeem
729 %                are normalised to the canonical English form)
730 % \<name>coord -- coordinate list (empty; filled by \computemodel)
731 % \<name>gridx -- x position in the graphic model (-1 = auto)
732 % \<name>gridy -- y position in the graphic model (-1 = auto)
733 % \<name>num   -- number with significance (via \num)
734 % \<name>qty   -- number + unit (via \qty)
735 % \<name>pre   -- engineering-prefix notation (via \qty)
736 % \<name>alias -- replaces the whole initial-value cell
737 %                (empty = default)
738 % \<name>aliasleft -- replaces the left symbol in the initial value
739 %                (empty = default)

```

```

740 % \<name>aliasright -- replaces the right number in the initial value
741 %                      (empty = default)
742 %
743 % Keys accepted in [#1]:
744 %   gridx, gridy      -- position in the graphic model
745 %   alias             -- replace the entire initial-value cell
746 %                      (in math mode)
747 %   aliasleft        -- replace just the left symbol
748 %   aliasright       -- replace just the right value
749 %
750 % The base value is registered in \g_defqty_names_seq so that
751 % \includeimage expands it automatically.
752 %
753 % With an empty start value (#3 blank), the base value is not
754 % numerically defined; using it issues a warning. Useful for
755 % auxiliary variables computed by \mrule.
756 %
757 % Example:
758 % \mvar{modM}{m}{80}{\kg}{2}{constant}
759 % \mvar{modFres}{F_{res}}{}{\N}{2}{aux}
760 % \mvar[aliasright=\cdots]{modX}{x}{0}{\m}{3}{stock}
761 % \mvar[alias={x \text{?}}]{modX}{x}{0}{\m}{3}{stock}
762 %
763 % Normalise the variable type (sixth \mvar argument) to its
764 % canonical English form. Accepts the canonical names
765 % {stock, constant, aux, system} and the Dutch synonyms
766 % {voorraad, constante, hulp, systeem}. An unknown value is left
767 % as-is and a warning is issued. The result is returned through
768 % \l__numodel_type_tl.
769 \tl_new:N \l__numodel_type_tl
770 \cs_new_protected:Npn \__numodel_normalize_type:n #1
771 {
772   \str_case:nnF {#1}
773   {
774     { stock      } { \tl_set:Nn \l__numodel_type_tl { stock      } }
775     { constant   } { \tl_set:Nn \l__numodel_type_tl { constant   } }
776     { aux        } { \tl_set:Nn \l__numodel_type_tl { aux        } }
777     { system     } { \tl_set:Nn \l__numodel_type_tl { system     } }
778     { voorraad   } { \tl_set:Nn \l__numodel_type_tl { stock      } }
779     { constante  } { \tl_set:Nn \l__numodel_type_tl { constant   } }
780     { hulp       } { \tl_set:Nn \l__numodel_type_tl { aux        } }
781     { systeem    } { \tl_set:Nn \l__numodel_type_tl { system     } }
782   }
783   {
784     \msg_warning:nne { numodel } { unknown-type } {#1}
785     \tl_set:Nn \l__numodel_type_tl {#1}
786   }
787 }
788
789 \makeatletter
790 \NewDocumentCommand{\mvar}{0}{ m m m m m m }{%
791   \typeout{MVAR-start:~#2}
792   % Parse optional keys (prefix, gridx, gridy, alias, aliasleft, aliasright,
793   % flowarrow-cloud-tip)
794   \tl_set:Nn \l__numodel_gridx_tl { -1 }
795   \tl_set:Nn \l__numodel_gridy_tl { -1 }
796   \tl_clear:N \l__numodel_alias_tl
797   \tl_clear:N \l__numodel_aliasleft_tl
798   \tl_clear:N \l__numodel_aliasright_tl
799   \tl_clear:N \l__numodel_cmd_prefix_tl
800   \tl_clear:N \l__numodel_mvar_flowcloud_tl
801   \typeout{MVAR-before-keys-set}
802   \keys_set:nn { numodel / mvar } {#1}
803   \typeout{MVAR-after-keys-set}
804   \__numodel_resolve_eff_prefix:
805   \typeout{MVAR-eff:~\l__numodel_eff_prefix_tl}

```

```

806 \_numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
807 { \_numodel_mvar_body:nnnnnn {#2}{#3}{#4}{#5}{#6}{#7} }%
808 \typeout{MVAR-done:~#2}
809 }
810 \makeatother
811
812 % Resolve the effective prefix from \l__numodel_cmd_prefix_tl.
813 \cs_new_protected:Npn \_numodel_resolve_eff_prefix:
814 {
815   \tl_if_empty:NTF \l__numodel_cmd_prefix_tl
816   { \tl_set_eq:NN \l__numodel_eff_prefix_tl \g_numodel_current_prefix_tl }
817   { \tl_set_eq:NN \l__numodel_eff_prefix_tl \l__numodel_cmd_prefix_tl }
818 }
819
820 % The \mvar body runs in the context of the right prefix (live state
821 % has already been swapped).
822 \cs_new_protected:Npn \_numodel_mvar_body:nnnnnn #1 #2 #3 #4 #5 #6
823 {
824   % Guard: current prefix must not be empty (\newmodelprefix required)
825   \tl_if_empty:NT \g_numodel_current_prefix_tl
826   { \msg_error:nn { numodel } { no-prefix } }
827   % Full name = current prefix + short name (#1)
828   \tl_set:Ne \l__numodel_fullname_tl { \g_numodel_current_prefix_tl #1 }
829   \typeout{MVAR~body~fullname:~\l__numodel_fullname_tl}
830   % Register in the per-prefix seq. The defqty registration is
831   % optional: it only fires when the user has loaded the project-
832   % specific 'defqty' system (used for worksheet expansion).
833   % Standalone, the package works without that seq.
834   \seq_gput_right:NV \g_mvar_names_seq \l__numodel_fullname_tl
835   \cs_if_exist:NT \g_defqty_names_seq
836   { \seq_gput_right:NV \g_defqty_names_seq \l__numodel_fullname_tl }
837   \directlua{ numodel.register(
838     "\g_numodel_current_prefix_tl",
839     "\l__numodel_fullname_tl" ) }
840   \cs_if_exist:cT { \l__numodel_fullname_tl }
841   { \msg_warning:nne { numodel } { redef } { \l__numodel_fullname_tl } }
842   \tl_if_blank:nTF {#3}
843   { \cs_gset:cpe { \l__numodel_fullname_tl } { \fp_eval:n { 0 } } }
844   {
845     \cs_gset:cpe { \l__numodel_fullname_tl } { \fp_eval:n {#3} }
846     \seq_gput_right:NV \g_mvar_start_seq \l__numodel_fullname_tl
847   }
848   \cs_gset:cpn { \l__numodel_fullname_tl text } {#2}
849   \cs_gset:cpn { \l__numodel_fullname_tl unit } { \unit{#4} }
850   \cs_gset:cpn { \l__numodel_fullname_tl unitraw } {#4}
851   \cs_gset:cpn { \l__numodel_fullname_tl sign } {#5}
852   \_numodel_normalize_type:n {#6}
853   \cs_gset:cpe { \l__numodel_fullname_tl type }
854   { \tl_use:N \l__numodel_type_tl }
855   \cs_gset:cpn { \l__numodel_fullname_tl min } { inf }
856   \cs_gset:cpn { \l__numodel_fullname_tl max } { -inf }
857   \cs_gset:cpe { \l__numodel_fullname_tl gridx } { \tl_use:N \l__numodel_gridx_tl }
858   \cs_gset:cpe { \l__numodel_fullname_tl gridy } { \tl_use:N \l__numodel_gridy_tl }
859   % Preserve the original user input so that \_numodel_build_graphic:
860   % can reset auto-placed positions to -1 on a second invocation.
861   \cs_gset:cpe { \l__numodel_fullname_tl gridxinit } { \tl_use:N \l__numodel_gridx_tl }
862   \cs_gset:cpe { \l__numodel_fullname_tl gridyinit } { \tl_use:N \l__numodel_gridy_tl }
863   % Pillar A - Lua-side meta for compute_layout (additive in A1).
864   % Detokenize text so that \luaescapestring works on bare chars.
865   \tl_set:Ne \l__numodel_scratch_tl { \detokenize {#2} }
866   \directlua{ numodel.set_meta(
867     "\g_numodel_current_prefix_tl",
868     "\l__numodel_fullname_tl",
869     { type = "\tl_use:N \l__numodel_type_tl",
870       text = "\luaescapestring{\l__numodel_scratch_tl}",
871       gridx = \tl_use:N \l__numodel_gridx_tl,

```

```

872         gridy = \tl_use:N \l__numodel_gridy_tl }) }
873     \cs_gset:cpe { \l__numodel_fullname_tl alias } { \exp_not:V \l__numodel_alias_tl }
874     \cs_gset:cpe { \l__numodel_fullname_tl aliasleft } { \exp_not:V \l__numodel_aliasleft_tl }
875     \cs_gset:cpe { \l__numodel_fullname_tl aliasright } { \exp_not:V \l__numodel_aliasright_tl }
876     \cs_gset:cpe { \l__numodel_fullname_tl flowcloud } { \exp_not:V \l__numodel_mvar_flowcloud_tl }
877     \tl_if_blank:nF {#3}
878     {
879         \exp_args:NV \NumodelDefNumCs \l__numodel_fullname_tl {\fp_eval:n {#3}} {#5}
880         \exp_args:NV \NumodelDefQtyCs \l__numodel_fullname_tl {\fp_eval:n {#3}} {#5} {#4}
881         \exp_args:NV \NumodelDefPreCs \l__numodel_fullname_tl {\fp_eval:n {#3}} {#5} {#4}
882     }
883 }
884
885 % =====
886 % Warning messages
887 % =====
888
889 \msg_new:nnn { numodel } { redef }
890 { Model~variable~'#1'~is~being~redefined. }
891 \msg_new:nnn { numodel } { empty-use }
892 { Model~variable~'#1'~has~no~start~value~and~is~used~before~assignment. }
893 \msg_new:nnn { numodel } { maxiter }
894 { computemodel~stopped~after~\int_use:N \g_numodel_maxiter_int ~iterations~
895   (safety~limit).~Check~stop~condition. }
896 \msg_new:nnn { numodel } { no-stop }
897 { computemodel:~no~stop~condition~defined.~Use~\token_to_str:N \mstop~space
898   before~\token_to_str:N \computemodel. }
899 \msg_new:nnn { numodel } { prefix-exists }
900 { Model~prefix~'#1'~is~already~registered.~
901   Use~\token_to_str:N \switchmodelprefix~space to~switch~to~an~existing~prefix. }
902 \msg_new:nnn { numodel } { prefix-unknown }
903 { Model~prefix~'#1'~is~not~registered.~
904   Use~\token_to_str:N \newmodelprefix~space to~create~it~first. }
905 \msg_new:nnn { numodel } { no-prefix }
906 { No~current~model~prefix.~
907   Call~\token_to_str:N \newmodelprefix{<name>}~space before~using~model~commands. }
908 \msg_new:nnn { numodel } { unknown-type }
909 { Unknown~variable~type~'#1'.~
910   Use~one~of~stock,~constant,~aux,~system~
911   (or~the~Dutch~aliases~voorraad,~constante,~hulp,~systeem). }
912
913 % =====
914 % Display helpers
915 % =====
916 % Translate computational expressions into syllabus-style display.
917 %
918 % Automatic translations:
919 % \modXxx      -> display name (via \<name>text)
920 % *            -> \cdot
921 % >= <=       -> \geqslant \leqslant
922 % sign(...)    -> SIGN(...) / Teken(...) in coachtaal
923 % abs(...)     -> ABS(...) / Abs(...) in coachtaal
924 % sqrt(...)    -> SQRT(...) / Sqrt(...) in coachtaal
925 % exp(...)     -> EXP(...) / Exp(...) in coachtaal
926 % ln(...)      -> LN(...) / Ln(...) in coachtaal
927 % sin(...)     -> SIN(...) / Sin(...) in coachtaal
928 % cos(...)     -> COS(...) / Cos(...) in coachtaal
929 % tan(...)     -> TAN(...) / Tan(...) in coachtaal
930 % asin(...)    -> ARCSIN(...) / Arcsin(...) in coachtaal
931 % acos(...)    -> ARCCOS(...) / Arccos(...) in coachtaal
932 % &&          -> AND / EN in coachtaal
933 % ||           -> OR / OF in coachtaal
934 % cond ? a : b -> IF cond THEN ... ELSE ... ENDIF (or coachtaal eq.)
935
936 \tl_new:N \l__numodel_display_tl
937 \tl_new:N \l__numodel_lhs_tl

```



```

938 \tl_new:N \l__numodel_rhs_tl
939 \tl_new:N \l__numodel_cond_tl
940 \tl_new:N \l__numodel_true_tl
941 \tl_new:N \l__numodel_false_tl
942
943 \cs_new_protected:Npn \__numodel_vars_to_display:N #1
944 {
945   \typeout{VTD-input:~\tl_to_str:N #1}
946   \typeout{VTD-seq:~\seq_use:Nn \g_mvar_names_seq {||}}
947   % Variable names -> display names
948   \seq_map_inline:Nn \g_mvar_names_seq
949   {
950     \typeout{VTD-iter:~##1}
951     \cs_if_exist:cT { ##1 text }
952     {
953       \tl_set:Nc \l_tmpa_tl { \use:c { ##1 text } }
954       \typeout{VTD-replace~\c{##1}~with~\l_tmpa_tl}
955       \regex_replace_all:nnN { \c{##1} } { \u{l_tmpa_tl} } #1
956     }
957   }
958   % Arithmetic operators
959   \regex_replace_all:nnN { \* } { \c{cdot} \x{20} } #1
960   \regex_replace_all:nnN { >= } { \c{geqslant} \x{20} } #1
961   \regex_replace_all:nnN { <= } { \c{leqslant} \x{20} } #1
962   % Functions (syllabus notation). Order matters: asin/acos must
963   % match before sin/cos, otherwise the inner sin/cos gets replaced
964   % first and the outer arc- prefix is left dangling.
965   \regex_replace_all:nnN { sign \(\ }
966   { \c{text}\cB{\u{g__numodel_kw_sign_tl}\cE}\( } #1
967   \regex_replace_all:nnN { abs \(\ }
968   { \c{text}\cB{\u{g__numodel_kw_abs_tl}\cE}\( } #1
969   \regex_replace_all:nnN { sqrt \(\ }
970   { \c{text}\cB{\u{g__numodel_kw_sqrt_tl}\cE}\( } #1
971   \regex_replace_all:nnN { exp \(\ }
972   { \c{text}\cB{\u{g__numodel_kw_exp_tl}\cE}\( } #1
973   \regex_replace_all:nnN { ln \(\ }
974   { \c{text}\cB{\u{g__numodel_kw_ln_tl}\cE}\( } #1
975   \regex_replace_all:nnN { asin \(\ }
976   { \c{text}\cB{\u{g__numodel_kw_asin_tl}\cE}\( } #1
977   \regex_replace_all:nnN { acos \(\ }
978   { \c{text}\cB{\u{g__numodel_kw_acos_tl}\cE}\( } #1
979   \regex_replace_all:nnN { sin \(\ }
980   { \c{text}\cB{\u{g__numodel_kw_sin_tl}\cE}\( } #1
981   \regex_replace_all:nnN { cos \(\ }
982   { \c{text}\cB{\u{g__numodel_kw_cos_tl}\cE}\( } #1
983   \regex_replace_all:nnN { \b tan \(\ }
984   { \c{text}\cB{\u{g__numodel_kw_tan_tl}\cE}\( } #1
985   % Logical operators (syllabus notation)
986   \regex_replace_all:nnN { \&\& }
987   { \c{text}\cB{\u{g__numodel_kw_and_tl}\cE}\( } #1
988   \regex_replace_all:nnN { \|\| }
989   { \c{text}\cB{\u{g__numodel_kw_or_tl}\cE}\( } #1
990 }
991
992 % Ternary detection: cond ? true_expr : false_expr
993 % Converted to: IF cond THEN lhs = true ELSE lhs = false ENDIF
994 % (or the coachtaal equivalent). Supports only flat (non-nested)
995 % ternaries.
996 \bool_new:N \l__numodel_is_ternary_bool
997
998 \cs_new_protected:Npn \__numodel_parse_ternary:nN #1 #2
999 {
1000   \bool_set_false:N \l__numodel_is_ternary_bool
1001   \regex_match:nnT { (.+) \? (.+) \: (.+) } {#1}
1002   {
1003     \bool_set_true:N \l__numodel_is_ternary_bool

```

```

1004 \tl_set:Nn \l__numodel_cond_tl {#1}
1005 \regex_replace_once:nnN { \s*(.+?) \s* \? .+ } { \1 } \l__numodel_cond_tl
1006 \tl_set:Nn \l__numodel_true_tl {#1}
1007 \regex_replace_once:nnN { .+? \? \s* (.+?) \s* \: .+ } { \1 } \l__numodel_true_tl
1008 \tl_set:Nn \l__numodel_false_tl {#1}
1009 \regex_replace_once:nnN { .+ \: \s* (.+?) \s* \Z } { \1 } \l__numodel_false_tl
1010 \__numodel_vars_to_display:N \l__numodel_cond_tl
1011 \__numodel_vars_to_display:N \l__numodel_true_tl
1012 \__numodel_vars_to_display:N \l__numodel_false_tl
1013 }
1014 }
1015
1016 % =====
1017 % \mrule[keys]{varname}{calculation}
1018 % =====
1019 % Declare a model rule.
1020 %
1021 % #1 (star) -- starred variant: multiline IF/THEN/ELSE/ENDIF
1022 % for ternary expressions
1023 % #2 (optional) -- keys: alias, aliasleft, aliasright
1024 % alias -- replaces the whole display row
1025 % aliasleft -- replaces the left-hand side
1026 % (symbol)
1027 % aliasright -- replaces the right-hand side
1028 % (calculation)
1029 % Use \cdots for omitted parts.
1030 % #3 -- name (string) of the left-hand variable
1031 % #4 -- calculation (with \modXxx macros and \fpeval
1032 % syntax)
1033 %
1034 % With alias or aliasright the ternary logic is skipped (display
1035 % shows the alias content verbatim, not IF/THEN/ELSE). Execution
1036 % always uses the calculation from #4.
1037 %
1038 % The calculation accepts all \fpeval operators:
1039 % +, -, *, /, ^, sign(), abs(), sin(), cos(), sqrt(), round()
1040 % Ternary: cond ? expr_true : expr_false
1041 % Logical: && (AND), || (OR), comparisons (<, >, <=, >=)
1042 %
1043 % The rule is stored for both display (\textmodel) and execution
1044 % (\computemodel).
1045 %
1046 % Examples:
1047 % \mrule{modFres}{\modM * \modA}
1048 % \mrule{modFw}{sign(\modV) * \modK * \modV^2}
1049 % \mrule{modA}{(\modT < \modTq) || (\modT > \modTdq) ? \modA + \modDa : \modA - \modDa}
1050 % \mrule[aliasright=\cdots]{modT}{\modT + \modDt}
1051 % \mrule[aliasleft=a_x]{modAx}{\modFgx / \modM}
1052 % \mrule[alias={\text{(hidden)}}]{modAy}{\modFgy / \modM}
1053
1054 \NewDocumentCommand{\mrule}{ s O{} m m }{%
1055 \typeout{MRULE~start:~#3}
1056 \bool_set_false:N \l__numodel_is_ternary_bool
1057 % Parse keys (prefix, alias, aliasleft, aliasright; gridx/gridy ignored)
1058 \tl_clear:N \l__numodel_alias_tl
1059 \tl_clear:N \l__numodel_aliasleft_tl
1060 \tl_clear:N \l__numodel_aliasright_tl
1061 \tl_clear:N \l__numodel_cmd_prefix_tl
1062 \keys_set:nn { numodel / mvar } {#2}
1063 \__numodel_resolve_eff_prefix:
1064 \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
1065 { \__numodel_mrule_body:nnn {#1}{#3}{#4} }%
1066 \typeout{MRULE~done:~#3}
1067 }
1068
1069 \cs_new_protected:Npn \__numodel_mrule_body:nnn #1 #2 #3

```

```

1070 {
1071   \int_gincr:N \g_mrule_counter_int
1072   \tl_set:Ne \l__numodel_fullname_tl { \g_numodel_current_prefix_tl #2 }
1073   % Store execution data (target = full name)
1074   \seq_gput_right:Nx \g_mrule_calc_seq
1075   { { \l__numodel_fullname_tl } { \exp_not:n {#3} } }
1076   % Left-hand side: aliasleft or default display name
1077   \tl_if_blank:VTF \l__numodel_aliasleft_tl
1078   { \tl_set:Ne \l__numodel_lhs_tl { \use:c { \l__numodel_fullname_tl text } } }
1079   { \tl_set_eq:NN \l__numodel_lhs_tl \l__numodel_aliasleft_tl }
1080   % Generate display
1081   \tl_if_blank:VTF \l__numodel_alias_tl
1082   {
1083     \tl_if_blank:VTF \l__numodel_aliasright_tl
1084     {
1085       % Automatic display generation (ternary or normal)
1086       \__numodel_parse_ternary:nN {#3} \l__numodel_display_tl
1087       \bool_if:NTF \l__numodel_is_ternary_bool
1088       {
1089         \IfBooleanTF {#1}
1090         {
1091           % Starred ternary -> multiline IF/THEN/ELSE/ENDIF
1092           \tl_set:Ne \l__numodel_display_tl
1093           {
1094             \__numodel_kwt:n {if}
1095             \exp_not:V \l__numodel_cond_tl
1096             \__numodel_kwt:n {then_n}
1097           }
1098           \seq_gput_right:NV \g_mrule_seq \l__numodel_display_tl
1099           \seq_gput_right:Nn \g_mrule_type_seq { rule }
1100           \tl_set:Ne \l__numodel_display_tl
1101           {
1102             \exp_not:n { \quad }
1103             \exp_not:V \l__numodel_lhs_tl \exp_not:n { \,=\, }
1104             \exp_not:V \l__numodel_true_tl
1105           }
1106           \seq_gput_right:NV \g_mrule_seq \l__numodel_display_tl
1107           \seq_gput_right:Nn \g_mrule_type_seq { cont }
1108           \seq_gput_right:Ne \g_mrule_seq { \__numodel_kwt:n {else_n} }
1109           \seq_gput_right:Nn \g_mrule_type_seq { cont }
1110           \tl_set:Ne \l__numodel_display_tl
1111           {
1112             \exp_not:n { \quad }
1113             \exp_not:V \l__numodel_lhs_tl \exp_not:n { \,=\, }
1114             \exp_not:V \l__numodel_false_tl
1115           }
1116           \seq_gput_right:NV \g_mrule_seq \l__numodel_display_tl
1117           \seq_gput_right:Nn \g_mrule_type_seq { cont }
1118           \seq_gput_right:Ne \g_mrule_seq { \__numodel_kwt:n {endif_n} }
1119           \seq_gput_right:Nn \g_mrule_type_seq { cont }
1120         }
1121         {
1122           % Unstarred ternary -> single-line
1123           \tl_set:Ne \l__numodel_display_tl
1124           {
1125             \__numodel_kwt:n {if}
1126             \exp_not:V \l__numodel_cond_tl
1127             \__numodel_kwt:n {then}
1128             \exp_not:V \l__numodel_lhs_tl \exp_not:n { \,=\, }
1129             \exp_not:V \l__numodel_true_tl
1130             \__numodel_kwt:n {else}
1131             \exp_not:V \l__numodel_lhs_tl \exp_not:n { \,=\, }
1132             \exp_not:V \l__numodel_false_tl
1133             \__numodel_kwt:n {endif}
1134           }
1135         }
1136       }
1137     }
1138   }

```

```

1136     }
1137     {
1138         % Ordinary assignment: lhs = rhs
1139         \tl_set:Nn \l__numodel_rhs_tl {#3}
1140         \__numodel_vars_to_display:N \l__numodel_rhs_tl
1141         \typeout{MRULE~rhs~after:~\tl_to_str:N \l__numodel_rhs_tl}
1142         \tl_set:Ne \l__numodel_display_tl
1143         {
1144             \exp_not:V \l__numodel_lhs_tl \exp_not:n { \,=\, }
1145             \exp_not:V \l__numodel_rhs_tl
1146         }
1147         \typeout{MRULE~display:~\tl_to_str:N \l__numodel_display_tl}
1148     }
1149 }
1150 {
1151     % aliasright: lhs = aliasright (no ternary processing)
1152     \tl_set:Ne \l__numodel_display_tl
1153     {
1154         \exp_not:V \l__numodel_lhs_tl \exp_not:n { \,=\, }
1155         \exp_not:V \l__numodel_aliasright_tl
1156     }
1157 }
1158 }
1159 {
1160     % alias: replace the whole row
1161     \tl_set:Ne \l__numodel_display_tl { \exp_not:V \l__numodel_alias_tl }
1162 }
1163 % For starred ternary everything is already pushed above
1164 \bool_if:nF { \l__numodel_is_ternary_bool && #1 }
1165 {
1166     \seq_gput_right:NV \g_mrulerule_seq \l__numodel_display_tl
1167     \seq_gput_right:Nn \g_mrulerule_type_seq { rule }
1168 }
1169 % Pillar A - Lua-side rule registration. Detokenize the raw
1170 % expression so Lua sees the macro names, not the evaluated
1171 % fp values. Lua handles both the dependency extraction
1172 % (build_deps) and the flow detection (classify_flows) at
1173 % \graphicmodel time.
1174 \tl_set:Ne \l__numodel_scratch_tl { \detokenize {#3} }
1175 \directlua{ numodel.add_rule(
1176     "\g_numodel_current_prefix_tl",
1177     "\l__numodel_fullname_tl",
1178     "\luaescapestring{\l__numodel_scratch_tl}",
1179     "\bool_if:NTF \l__numodel_is_ternary_bool { ternary } { calc }") }
1180 }
1181
1182 % =====
1183 % \mruletext{free text}
1184 % =====
1185 % Adds a free-text row in the model table. Useful for structure
1186 % that does not fit as \mrule, e.g.:
1187 % \mruletext{\text{IF } t < T/4 \text{ THEN}}
1188 % \mruletext{\quad a = a + da}
1189 % \mruletext{\text{ENDIF}}
1190 %
1191 % NOT executed by \computemodel (display only).
1192
1193 \NewDocumentCommand{\mruletext}{0}{ m }{%
1194     \tl_clear:N \l__numodel_cmd_prefix_tl
1195     \keys_set:nn { numodel / cmd } {#1}
1196     \__numodel_resolve_eff_prefix:
1197     \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
1198     {
1199         \int_gincr:N \g_mrulerule_counter_int
1200         \seq_gput_right:Nn \g_mrulerule_seq {#2}
1201         \seq_gput_right:Nn \g_mrulerule_type_seq { rule }

```

```

1202     }
1203 }
1204
1205 % =====
1206 % \mstop[keys]{condition}
1207 % \mstop*[keys]{condition}    (star reserved; currently identical)
1208 % =====
1209 % Declare the model's stop condition.
1210 % Display: "IF condition THEN STOP ENDIF"
1211 % Execution: the model stops when the condition is true (evaluates to 1).
1212 %
1213 % The condition uses \fpeval syntax with \mvar variables:
1214 %   \mstop{\modT >= \modTmax}
1215 %   \mstop{\modV <= 0}
1216 %
1217 % Supported keys:
1218 %   alias={...} -- replaces the whole display row (execution
1219 %                 remains intact)
1220 % aliasleft/aliasright are not (yet) supported for \mstop.
1221
1222 \tl_new:N \l__numodel_stop_tl
1223 \NewDocumentCommand{\mstop}{s O{} m }{%
1224   \typeout{MSTOP~start}
1225   % Parse keys (only alias is honoured; prefix via key resolver)
1226   \tl_clear:N \l__numodel_alias_tl
1227   \tl_clear:N \l__numodel_aliasleft_tl
1228   \tl_clear:N \l__numodel_aliasright_tl
1229   \tl_clear:N \l__numodel_cmd_prefix_tl
1230   \keys_set:nn { numodel / mvar } {#2}
1231   \__numodel_resolve_eff_prefix:
1232   \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
1233   { \__numodel_mstop_body:n {#3} }%
1234   \typeout{MSTOP~done}
1235 }
1236
1237 \cs_new_protected:Npn \__numodel_mstop_body:n #1
1238 {
1239   \int_gincr:N \g_mruler_counter_int
1240   % Store expression for execution (always the actual condition)
1241   \tl_gset:Nn \g_numodel_stop_expr_tl {#1}
1242   % Generate display
1243   \tl_if_blank:VTF \l__numodel_alias_tl
1244   {
1245     \tl_set:Nn \l__numodel_stop_tl {#1}
1246     \__numodel_vars_to_display:N \l__numodel_stop_tl
1247     \tl_set:Nn \l__numodel_display_tl
1248     {
1249       \__numodel_kwt:n {if}
1250       \exp_not:V \l__numodel_stop_tl
1251       \__numodel_kwt:n {then_n}
1252       \__numodel_kwt:n {stop}
1253       \__numodel_kwt:n {endif_n}
1254     }
1255   }
1256   {
1257     \tl_set:Nn \l__numodel_display_tl { \exp_not:V \l__numodel_alias_tl }
1258   }
1259   \seq_gput_right:NV \g_mruler_seq \l__numodel_display_tl
1260   \seq_gput_right:Nn \g_mruler_type_seq { rule }
1261 }
1262
1263 % =====
1264 % \textmodel
1265 % =====
1266 % Builds a tabular with numbered model rules on the left and initial
1267 % values on the right. Can be placed anywhere, e.g. in a subfigure

```

```

1268 % next to a graphic model.
1269
1270 \tl_new:N \g__numodel_table_tl
1271 \int_new:N \l__numodel_row_int
1272 \int_new:N \l__numodel_dispnum_int
1273 \int_new:N \l__numodel_startrow_int
1274 \int_new:N \l__numodel_numrules_int
1275 \int_new:N \l__numodel_numstarts_int
1276
1277 % Emit one initial-value cell with alias-key support.
1278 % - \<name>alias not empty -> replaces whole cell (inside $...$)
1279 % - \<name>aliasleft not empty -> replaces left symbol, else
1280 % \<name>text
1281 % - \<name>aliasright not empty -> replaces right value, else
1282 % \<name>qty (units=true) or
1283 % \<name>num (units=false)
1284 \cs_new_protected:Npn \__numodel_emit_startcell:n #1
1285 {
1286   \tl_if_blank:eTF { \use:c { #1 alias } }
1287   {
1288     \tl_gput_right:Nn \g__numodel_table_tl { $ }
1289     \tl_if_blank:eTF { \use:c { #1 aliasleft } }
1290     {
1291       \tl_gput_right:Ne \g__numodel_table_tl
1292       { \exp_not:e { \use:c { #1 text } } }
1293     }
1294     {
1295       \tl_gput_right:Ne \g__numodel_table_tl
1296       { \exp_not:e { \use:c { #1 aliasleft } } }
1297     }
1298     \tl_gput_right:Nn \g__numodel_table_tl { = }
1299     \tl_if_blank:eTF { \use:c { #1 aliasright } }
1300     {
1301       % Emit \<name>qty or \<name>num as a token name without
1302       % expanding it; siunitx macros must only expand at
1303       % \tl_use:N time, in the correct tabular context.
1304       \bool_if:NTF \g__numodel_units_bool
1305       {
1306         \tl_gput_right:Nx \g__numodel_table_tl
1307         { \exp_not:c { #1 qty } }
1308       }
1309       {
1310         \tl_gput_right:Nx \g__numodel_table_tl
1311         { \exp_not:c { #1 num } }
1312       }
1313     }
1314     {
1315       \tl_gput_right:Ne \g__numodel_table_tl
1316       { \exp_not:e { \use:c { #1 aliasright } } }
1317     }
1318     \tl_gput_right:Nn \g__numodel_table_tl { $ }
1319   }
1320   {
1321     \tl_gput_right:Nn \g__numodel_table_tl { $ }
1322     \tl_gput_right:Ne \g__numodel_table_tl
1323     { \exp_not:e { \use:c { #1 alias } } }
1324     \tl_gput_right:Nn \g__numodel_table_tl { $ }
1325   }
1326 }
1327
1328 \cs_new_protected:Npn \__numodel_build_table:
1329 {
1330   \typeout{BUILD_TABLE~rules:~\seq_use:Nn \g_mrule_seq {||}}
1331   \typeout{BUILD_TABLE~types:~\seq_use:Nn \g_mrule_type_seq {||}}
1332   \int_set:Nn \l__numodel_numrules_int { \seq_count:N \g_mrule_seq }
1333   \int_set:Nn \l__numodel_numstarts_int { \seq_count:N \g_mvar_start_seq }

```

```

1334 \typeout{BUILD_TABLE~numrules:~\int_use:N \l__numodel_numrules_int}
1335 \typeout{BUILD_TABLE~numstarts:~\int_use:N \l__numodel_numstarts_int}
1336 \int_zero:N \l__numodel_row_int
1337 \int_zero:N \l__numodel_dispnum_int
1338 \int_zero:N \l__numodel_startrow_int
1339 \tl_gset:Nn \g__numodel_table_tl
1340 { \begin{tabular}{r|l|l} & \textbf{ }
1341 \tl_gput_right:Ne \g__numodel_table_tl
1342 { { \__numodel_kw:n {th_model} } }
1343 \tl_gput_right:Nn \g__numodel_table_tl
1344 { & \textbf{ }
1345 \tl_gput_right:Ne \g__numodel_table_tl
1346 { { \__numodel_kw:n {th_initvals} } }
1347 \tl_gput_right:Nn \g__numodel_table_tl
1348 { \\\hline }
1349 \typeout{BUILD_TABLE~before-step-table:~\tl_to_str:N \g__numodel_table_tl}
1350 \int_step_inline:nn { \l__numodel_numrules_int }
1351 {
1352 \typeout{BUILD_TABLE~iter-row:~\int_use:N \l__numodel_row_int}
1353 \int_incr:N \l__numodel_row_int
1354 \int_incr:N \l__numodel_startrow_int
1355 \typeout{BUILD_TABLE~row:~\int_use:N \l__numodel_row_int~type:~\seq_item:Nn \g_mruler_type_seq
1356 % Check type: rule -> show number, cont -> blank
1357 \str_if_eq:eeTF
1358 { \seq_item:Nn \g_mruler_type_seq { \l__numodel_row_int } }
1359 { cont }
1360 {
1361 % Continuation rows: no number
1362 \tl_gput_right:Ne \g__numodel_table_tl
1363 {
1364 \exp_not:n { \rule{0pt}{2.6ex} }
1365 \exp_not:n { & $ }
1366 \exp_not:e { \seq_item:Nn \g_mruler_seq { \l__numodel_row_int } }
1367 \exp_not:n { $ & }
1368 }
1369 }
1370 {
1371 % Numbered row
1372 \int_incr:N \l__numodel_dispnum_int
1373 \typeout{BUILD_TABLE~emit~rule~num:~\int_use:N \l__numodel_dispnum_int~content:~\seq_item
1374 \tl_gput_right:Ne \g__numodel_table_tl
1375 {
1376 \exp_not:n { \rule{0pt}{2.6ex} }
1377 \int_use:N \l__numodel_dispnum_int
1378 \exp_not:n { & $ }
1379 \exp_not:e { \seq_item:Nn \g_mruler_seq { \l__numodel_row_int } }
1380 \exp_not:n { $ & }
1381 }
1382 \typeout{BUILD_TABLE~after-emit-table-tail:~\tl_tail:N \g__numodel_table_tl}
1383 }
1384 % Initial-value column (independent of rule/cont)
1385 \int_compare:nNnTF { \l__numodel_startrow_int } > { \l__numodel_numstarts_int }
1386 {
1387 \tl_gput_right:Nn \g__numodel_table_tl { \\\ }
1388 }
1389 {
1390 \exp_args:Ne \__numodel_emit_startcell:n
1391 { \seq_item:Nn \g_mvar_start_seq { \l__numodel_startrow_int } }
1392 \tl_gput_right:Nn \g__numodel_table_tl { \\\ }
1393 }
1394 }
1395 \int_while_do:nNnn { \l__numodel_startrow_int } < { \l__numodel_numstarts_int }
1396 {
1397 \int_incr:N \l__numodel_startrow_int
1398 \tl_gput_right:Nn \g__numodel_table_tl { & & }
1399 \exp_args:Ne \__numodel_emit_startcell:n

```

```

1400         { \seq_item:Nn \g_mvar_start_seq { \l__numodel_startrow_int } }
1401         \tl_gput_right:Nn \g__numodel_table_tl { \\ }
1402     }
1403     \tl_gput_right:Nn \g__numodel_table_tl { \end{tabular} }
1404 }
1405
1406 \NewDocumentCommand{\textmodel}{O{}}{%
1407     \typeout{TEXTMODEL~start}
1408     \tl_clear:N \l__numodel_cmd_prefix_tl
1409     \tl_clear:N \l__numodel_cmd_units_tl
1410     \keys_set:nn { numodel / cmd } {#1}
1411     \__numodel_resolve_eff_prefix:
1412     % Temporary units override: save the global value, apply the key
1413     % value before build, restore afterwards. This way
1414     % \textmodel[units=false] flips one render without touching the
1415     % global \numodelsetup state.
1416     \bool_set_eq:NN \l__numodel_saved_units_bool \g__numodel_units_bool
1417     \tl_if_empty:NF \l__numodel_cmd_units_tl
1418     {
1419         \str_if_eq:VnTF \l__numodel_cmd_units_tl { true }
1420         { \bool_gset_true:N \g__numodel_units_bool }
1421         { \bool_gset_false:N \g__numodel_units_bool }
1422     }
1423     \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
1424     {
1425         \group_begin:
1426         \__numodel_apply_dsep:
1427         \__numodel_build_table:
1428         \typeout{TEXTMODEL~table:~\tl_to_str:N \g__numodel_table_tl}
1429         \tl_use:N \g__numodel_table_tl
1430         \group_end:
1431     }%
1432     \bool_gset_eq:NN \g__numodel_units_bool \l__numodel_saved_units_bool
1433     \typeout{TEXTMODEL~done}
1434 }
1435 \bool_new:N \l__numodel_saved_units_bool
1436
1437 % =====
1438 % \computemodel
1439 % =====
1440 % Run the model with the Euler method:
1441 % 1. Record all variable values in Lua
1442 % 2. Check stop condition (\mstop)
1443 % 3. Execute every \mrule rule (in declaration order)
1444 % 4. Repeat until stop or safety limit (default 20000 iterations)
1445 %
1446 % Requires: at least one \mstop and at least one \mrule.
1447 % Performance: ~440 steps in ~4s, ~20000 steps in ~60s.
1448
1449 \cs_new_protected:Npn \__numodel_exec_rule:nn #1 #2
1450 {
1451     \cs_gset:cpe {#1} { \fp_eval:n {#2} }
1452 }
1453
1454 % Record all current variable values in Lua (0(1) per variable).
1455 \cs_new_protected:Npn \__numodel_lua_record_all:
1456 {
1457     \seq_map_inline:Nn \g_mvar_names_seq
1458     {
1459         \directlua{ numodel.record(
1460             "\g_numodel_current_prefix_tl", "##1", \use:c{##1}) }
1461     }
1462     \directlua{ numodel.end_step("\g_numodel_current_prefix_tl") }
1463 }
1464
1465 % Set min/max TeX macros from Lua data after the simulation finishes.

```



```

1466 \cs_new_protected:Npn \__numodel_set_minmax_from_lua:
1467 {
1468   \seq_map_inline:Nn \g_mvar_names_seq
1469   {
1470     \cs_gset:cpe { ##1 min }
1471     { \directlua{ numodel.get_min("\g_numodel_current_prefix_tl", "##1") } }
1472     \cs_gset:cpe { ##1 max }
1473     { \directlua{ numodel.get_max("\g_numodel_current_prefix_tl", "##1") } }
1474   }
1475 }
1476
1477
1478 \NewDocumentCommand{\computemodel}{O{}}{%
1479   \typeout{COMPUTE~start}
1480   \tl_clear:N \l__numodel_cmd_prefix_tl
1481   \keys_set:nn { numodel / cmd } {#1}
1482   \__numodel_resolve_eff_prefix:
1483   \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
1484   { \__numodel_compute_body: }%
1485   \typeout{COMPUTE~done}
1486 }
1487
1488 \cs_new_protected:Npn \__numodel_compute_body:
1489 {
1490   \tl_if_empty:NT \g_numodel_stop_expr_tl
1491   { \msg_error:nn { numodel } { no-stop } }
1492   \int_gzero:N \g_numodel_steps_int
1493   % Initialise Lua storage for this prefix
1494   \directlua{ numodel.init("\g_numodel_current_prefix_tl") }
1495   % Main loop
1496   \bool_gset_false:N \g_tmpa_bool
1497   \bool_do_until:Nn \g_tmpa_bool
1498   {
1499     \__numodel_lua_record_all:
1500     \int_gincr:N \g_numodel_steps_int
1501     \int_compare:nNnT
1502     { \fp_eval:n { \g_numodel_stop_expr_tl } } = { 1 }
1503     { \bool_gset_true:N \g_tmpa_bool }
1504     \bool_if:NF \g_tmpa_bool
1505     {
1506       \seq_map_inline:Nn \g_mrule_calc_seq
1507       { \__numodel_exec_rule:nn ##1 }
1508     }
1509     \int_compare:nNnT
1510     { \g_numodel_steps_int } > { \g_numodel_maxiter_int }
1511     {
1512       \bool_gset_true:N \g_tmpa_bool
1513       \msg_warning:nn { numodel } { maxiter }
1514     }
1515   }
1516   \__numodel_set_minmax_from_lua:
1517 }
1518
1519 % On-demand coordinates from Lua (after \computemodel). Arguments
1520 % are SHORT names; the current prefix is prepended automatically.
1521 % See \mcoordsp for the form with an explicit prefix.
1522 %
1523 % Fully expandable (a plain \def around \directlua, no xparse
1524 % wrapper). Works directly inside \addplot coordinates
1525 % {\mcoords{T}{V}} and pgfplots' math-expression parser, without
1526 % pre-expansion via \edef.
1527 \cs_new:Npn \mcoords #1#2
1528 {
1529   \directlua{ numodel.get_coords(
1530     "\g_numodel_current_prefix_tl",
1531     "\g_numodel_current_prefix_tl" .. "#1",

```

```

1532     "\g_numodel_current_prefix_tl" .. "#2") }
1533 }
1534
1535 % Variant with explicit prefix as first argument (instead of an
1536 % optional argument, so the macro stays fully expandable).
1537 \cs_new:Npn \mcoordsp #1#2#3
1538 {
1539     \directlua{ numodel.get_coords(
1540         "#1", "#1" .. "#2", "#1" .. "#3") }
1541 }
1542
1543 % Value of variable #1 at step #2 (0-based). The variable name is
1544 % SHORT (current prefix is prepended automatically). See \mstepp for
1545 % the form with an explicit prefix.
1546 %
1547 % Example -- tangent line at step 0:
1548 %     \addplot[domain=0:\modTmax]
1549 %         {\mstep{V}{0} + \mstep{A}{1} * (x - \mstep{T}{0})};
1550 \cs_new:Npn \mstep #1#2
1551 {
1552     \directlua{ numodel.get_step(
1553         "\g_numodel_current_prefix_tl",
1554         "\g_numodel_current_prefix_tl" .. "#1", #2) }
1555 }
1556
1557 \cs_new:Npn \mstepp #1#2#3
1558 {
1559     \directlua{ numodel.get_step("#1", "#1" .. "#2", #3) }
1560 }
1561
1562 % =====
1563 % \modelreset -- REMOVED
1564 % =====
1565 % \modelreset no longer exists. Use \newmodelprefix{<prefix>} to set
1566 % up a new model and \switchmodelprefix{<prefix>} to switch between
1567 % existing ones. Namespaces are additive -- variables from earlier
1568 % models remain available.
1569
1570 % =====
1571 % \graphicmodel -- Forrester diagram from \mvar/\mrule data
1572 % =====
1573 % Builds a tikzpicture with:
1574 %   - stock, valve, aux and const nodes based on type and gridx/gridy
1575 %   - flow arrows from valve to stock (via \flowarrow)
1576 %   - causal arrows from the dependency graph
1577 %
1578 % Positioning: automatic (auto-layout) or manual via
1579 %   \mvar[gridx=N, gridy=N]{...}. Mixed mode is supported.
1580 % Auto-layout places stocks+valves at gridy=0, aux at gridy=1, and
1581 % constants at gridy=2. Variables of type system are skipped.
1582 % Auxiliary variables that act as inflow are placed as a valve
1583 % (not as aux).
1584
1585 \tl_new:N \g_numodel_graphic_tl
1586 % Lua-populated cache (filled by numodel.tex_writeback at the start of
1587 % each \graphicmodel call, consumed by the emit helpers below).
1588 \prop_new:N \l_numodel_valve_for_prop % aux/const -> stock (inflow)
1589 \prop_new:N \l_numodel_outvalve_for_prop % aux/const -> stock (outflow)
1590 \prop_new:N \l_numodel_between_valve_prop % aux/const -> source_stock (between-flow)
1591 \prop_new:N \l_numodel_between_target_prop % aux/const -> target_stock (between-flow)
1592 \prop_new:N \l_numodel_stock_valve_prop % stock -> source stock (stock-as-flow)
1593 \prop_new:N \l_numodel_stock_phantom_valve_prop % stock -> source stock (phantom flow)
1594 % For diagram-style=forrester|edu: separate valve gridx position next
1595 % to the stock; the variable's own gridx/gridy keeps the natural
1596 % position at gridy=1 (aux) or gridy=2 (constant). vpos_y is always 0
1597 % so it lives implicitly in the emit helpers.

```

```

1598 \prop_new:N \l__numodel_vpos_x_prop      % varname -> valve gridx
1599 \prop_new:N \l__numodel_vpos_y_prop      % varname -> valve gridy (used when gridmaxx wrap shifted
1600 \tl_new:N   \l__numodel_flows_tl         % deferred flow arrows
1601 \tl_new:N   \l__numodel_valves_tl        % deferred valve nodes (drawn on top)
1602 \tl_new:N   \l__numodel_late_causals_tl  % causals pointing at valves
1603
1604 % --- Resolved render settings (set per-\graphicmodel call) ---
1605 % These hold the effective values (after resolving diagram-style
1606 % defaults and explicit global/local overrides) used by the emit
1607 % helpers and by the public \flowarrow / \flowoutarrow macros.
1608 \tl_new:N   \l__numodel_flowarrow_eff_tl  % "hollow" or "filled"
1609 \tl_new:N   \l__numodel_valve_eff_tl      % "valve" | "circle" | "edu"
1610 \tl_new:N   \l__numodel_flowcloud_global_tl % "true" or "false" (global default)
1611
1612 % Resolve flowarrow-style: explicit global key wins; otherwise
1613 % diagram-style picks the default (forrester -> hollow,
1614 % tight|edu -> filled).
1615 \cs_new_protected:Npn \__numodel_resolve_flowarrow:
1616 {
1617   \tl_if_empty:NTF \g__numodel_flowarrow_style_tl
1618   {
1619     \str_if_eq:VnTF \g__numodel_diagram_style_tl { forrester }
1620     { \tl_set:Nn \l__numodel_flowarrow_eff_tl { hollow } }
1621     { \tl_set:Nn \l__numodel_flowarrow_eff_tl { filled } }
1622   }
1623   { \tl_set_eq:NN \l__numodel_flowarrow_eff_tl
1624     \g__numodel_flowarrow_style_tl }
1625 }
1626
1627 % Resolve valve-style: explicit global key wins; otherwise
1628 % diagram-style picks the default (forrester -> valve,
1629 % tight|edu -> edu).
1630 \cs_new_protected:Npn \__numodel_resolve_valve:
1631 {
1632   \tl_if_empty:NTF \g__numodel_valve_style_tl
1633   {
1634     \str_if_eq:VnTF \g__numodel_diagram_style_tl { forrester }
1635     { \tl_set:Nn \l__numodel_valve_eff_tl { valve } }
1636     { \tl_set:Nn \l__numodel_valve_eff_tl { edu } }
1637   }
1638   { \tl_set_eq:NN \l__numodel_valve_eff_tl
1639     \g__numodel_valve_style_tl }
1640 }
1641
1642 % Resolve the global flowarrow-cloud-tip default (no per-stock
1643 % override yet; that's added on top in \__numodel_eff_flowcloud:n).
1644 \cs_new_protected:Npn \__numodel_resolve_flowcloud:
1645 {
1646   \tl_if_empty:NTF \g__numodel_flowcloud_tl
1647   {
1648     \str_if_eq:VnTF \g__numodel_diagram_style_tl { forrester }
1649     { \tl_set:Nn \l__numodel_flowcloud_global_tl { true } }
1650     { \tl_set:Nn \l__numodel_flowcloud_global_tl { false } }
1651   }
1652   { \tl_set_eq:NN \l__numodel_flowcloud_global_tl
1653     \g__numodel_flowcloud_tl }
1654 }
1655
1656 % Per-stock flowcloud lookup. #1 = stock varname. Sets
1657 % \l__numodel_flowcloud_eff_tl to "true" or "false". Order: per-mvar
1658 % override on the stock (via [flowarrow-cloud-tip=...]) wins over the
1659 % global default.
1660 \tl_new:N   \l__numodel_flowcloud_eff_tl
1661 \cs_new_protected:Npn \__numodel_eff_flowcloud:n #1
1662 {
1663   \tl_set:Nn \l__numodel_flowcloud_eff_tl { \use:c { #1 flowcloud } }

```

```

1664 \tl_if_empty:NT \l__numodel_flowcloud_eff_tl
1665 { \tl_set_eq:NN \l__numodel_flowcloud_eff_tl
1666 \l__numodel_flowcloud_global_tl }
1667 }
1668
1669 % Sets the public \nmflowbody and \nmflowtip macros that the
1670 % \flowarrow / \flowoutarrow / \flowbetweenarrow macros expand to.
1671 % Reads the resolved flowarrow-style.
1672 \cs_new_protected:Npn \__numodel_apply_flowarrow_style:
1673 {
1674 \str_if_eq:VnTF \l__numodel_flowarrow_eff_tl { hollow }
1675 {
1676 \cs_gset:Npn \nmflowbody { flowpipe-hollow }
1677 \cs_gset:Npn \nmflowtip { flowpipe-hollow-tip }
1678 \cs_gset:Npn \nmflowtipcloudin { flowpipe-hollow-cloudin }
1679 \cs_gset:Npn \nmflowtipcloudout { flowpipe-hollow-cloudout }
1680 }
1681 {
1682 \cs_gset:Npn \nmflowbody { flowpipe-filled }
1683 \cs_gset:Npn \nmflowtip { flowpipe-filled-tip }
1684 \cs_gset:Npn \nmflowtipcloudin { flowpipe-filled-cloudin }
1685 \cs_gset:Npn \nmflowtipcloudout { flowpipe-filled-cloudout }
1686 }
1687 }
1688
1689 % Sets the tikz node-style name for the valve and the boolean that
1690 % controls whether the valve carries the variable's display label.
1691 % valve-style=valve -> [valve-forrester], no label
1692 % valve-style=circle -> [valve-circle], no label
1693 % valve-style=edu -> [valve-edu], with label
1694 \tl_new:N \l__numodel_valve_node_tl
1695 \bool_new:N \l__numodel_valve_label_bool
1696 \cs_new_protected:Npn \__numodel_apply_valve_style:
1697 {
1698 \str_case:VnF \l__numodel_valve_eff_tl
1699 {
1700 { valve }
1701 {
1702 \tl_set:Nn \l__numodel_valve_node_tl { valve-forrester }
1703 \bool_set_false:N \l__numodel_valve_label_bool
1704 }
1705 { circle }
1706 {
1707 \tl_set:Nn \l__numodel_valve_node_tl { valve-circle }
1708 \bool_set_false:N \l__numodel_valve_label_bool
1709 }
1710 { edu }
1711 {
1712 \tl_set:Nn \l__numodel_valve_node_tl { valve-edu }
1713 \bool_set_true:N \l__numodel_valve_label_bool
1714 }
1715 }
1716 {
1717 \tl_set:Nn \l__numodel_valve_node_tl { valve-edu }
1718 \bool_set_true:N \l__numodel_valve_label_bool
1719 }
1720 }
1721
1722 % --- Diagram-style mode flag ---
1723 % True at diagram-style=forrester|edu; consulted by \__numodel_place_node
1724 % to decide between single-emit and natural+phantom-valve double-emit.
1725 % Set in \__numodel_build_graphic from \g__numodel_diagram_style_tl.
1726 \bool_new:N \l__numodel_keep_natural_bool
1727
1728
1729 \cs_new_protected:Npn \__numodel_build_graphic:

```

```

1730 {
1731   \typeout{BUILD:~step-0~reset~auto~positions}
1732   % --- Reset gridx/gridy to the original user input ---
1733   % Auto-layout writes positions to var.gridx/gridy. On a second
1734   % \graphicmodel call those would, without reset, be treated as
1735   % "manually placed". The initial values are saved in
1736   % gridxinit/gridyinit by \mvar -- copy them back.
1737   \seq_map_inline:Nn \g_mvar_names_seq
1738   {
1739     \cs_gset:cpe { ##1 gridx } { \use:c { ##1 gridxinit } }
1740     \cs_gset:cpe { ##1 gridy } { \use:c { ##1 gridyinit } }
1741   }
1742   % Diagram-style mode: forrester|edu keep aux/const at their natural
1743   % gridy with a phantom valve next to the stock; tight collapses the
1744   % aux/const onto the valve position. Consumed by \__numodel_place_node
1745   % to dispatch between single-emit and natural+phantom emit.
1746   \bool_set_false:N \l__numodel_keep_natural_bool
1747   \str_if_eq:VnT \g__numodel_diagram_style_tl { forrester }
1748   { \bool_set_true:N \l__numodel_keep_natural_bool }
1749   \str_if_eq:VnT \g__numodel_diagram_style_tl { edu }
1750   { \bool_set_true:N \l__numodel_keep_natural_bool }
1751   \typeout{BUILD:~step-1~lua~layout~writeback}
1752   % Pillar A - Lua is the single source of truth for flow detection
1753   % and auto-layout. The writeback clears and fills \<var>gridx/gridy
1754   % and the flow-props that downstream emitters (place_node,
1755   % flow-builders, emit_natural_and_phantom, emit_stock_valve) read.
1756   \__numodel_lua_layout_writeback:
1757   \typeout{BUILD:~step-2~tikzpicture}
1758   % --- Build tikzpicture ---
1759   % Render order matters: each segment overlays the previous one,
1760   % so we draw flows first (one continuous arrow per flow), then
1761   % the valve nodes on top (with white fill so they cover the
1762   % section of the arrow that lies underneath), and finally the
1763   % causal arrows that point at those valves.
1764   \tl_gclear:N \g__numodel_graphic_tl
1765   \tl_clear:N \l__numodel_flows_tl
1766   \tl_clear:N \l__numodel_valves_tl
1767   \tl_clear:N \l__numodel_late_causals_tl
1768   \tl_gput_right:Nn \g__numodel_graphic_tl
1769   { \begin{tikzpicture}[gridscale] }
1770   % Nodes (stocks, aux, const, clouds; flows -> flows_tl,
1771   % valves -> valves_tl)
1772   \seq_map_inline:Nn \g_mvar_names_seq
1773   { \typeout{NODE:~##1} \__numodel_place_node:n {##1} }
1774   \typeout{BUILD:~step-5~stock-valve~nodes}
1775   % Stock-as-flow valve nodes
1776   \prop_map_inline:Nn \l__numodel_stock_valve_prop
1777   {
1778     % ##1 = stock (e.g. modH), ##2 = source stock (e.g. modV)
1779     % Skip entries that are not real stock-valves (e.g. __svgx keys)
1780     \tl_if_in:nnF {##1} { __sv }
1781     { \__numodel_emit_stock_valve:nn {##1} {##2} }
1782   }
1783   % Stock-as-rate phantom-valve nodes (source stock has no matching
1784   % outflow term, so the inflow gets a cloud at the open end)
1785   \prop_map_inline:Nn \l__numodel_stock_phantom_valve_prop
1786   {
1787     \tl_if_in:nnF {##1} { __sv }
1788     { \__numodel_emit_stock_phantom_valve:nn {##1} {##2} }
1789   }
1790   \typeout{BUILD:~step-6~flow~arrows}
1791   % Flow arrows (one segment each; drawn underneath the valves)
1792   \tl_gput_right:NV \g__numodel_graphic_tl \l__numodel_flows_tl
1793   \typeout{BUILD:~step-6a~valve~nodes}
1794   % Valve nodes (white-filled, drawn ON TOP of the flow arrows)
1795   \tl_gput_right:NV \g__numodel_graphic_tl \l__numodel_valves_tl

```

```

1796 \typeout{BUILD:~step-6b+7~causal~arrows~(Lua)}
1797 % A2: causal arrows via Lua - emit_causals demultiplexes on
1798 % tgt_is_valve and pushes to \g__numodel_graphic_tl or
1799 % \l__numodel_late_causals_tl respectively. Afterwards append
1800 % late-causals to graphic_tl once: that covers both the pushes
1801 % from the flow-builders (step 6/6a) and those from emit_causals.
1802 \__numodel_lua_emit_causals:
1803 \tl_gput_right:NV \g__numodel_graphic_tl \l__numodel_late_causals_tl
1804 \typeout{BUILD:~step-8~done}
1805 \tl_gput_right:Nn \g__numodel_graphic_tl
1806 { \end{tikzpicture} }
1807 }
1808
1809 % --- Node placement ---
1810 \cs_new_protected:Npn \__numodel_place_node:n #1
1811 {
1812   \bool_set_true:N \l_tmpa_bool
1813   \tl_set:Nc \l__numodel_scratch_tl { \use:c { #1 type } }
1814   \exp_args:NV \str_if_eq:nnT \l__numodel_scratch_tl { system }
1815     { \bool_set_false:N \l_tmpa_bool }
1816   \bool_if:NT \l_tmpa_bool
1817     {
1818       \int_compare:nNnT { \use:c { #1 gridx } } = { -1 }
1819       { \bool_set_false:N \l_tmpa_bool }
1820     }
1821   \bool_if:NT \l_tmpa_bool
1822     {
1823       \tl_set:Nc \l__numodel_tmp_tl { \use:c { #1 text } }
1824       % With forrester|edu, valve vars get a dual emission:
1825       % the natural aux/const node + a phantom valve next to the stock.
1826       \bool_set_false:N \l_tmpb_bool % is this a valve var?
1827       \prop_if_in:NnT \l__numodel_valve_for_prop {#1}
1828       { \bool_set_true:N \l_tmpb_bool }
1829       \prop_if_in:NnT \l__numodel_outvalve_for_prop {#1}
1830       { \bool_set_true:N \l_tmpb_bool }
1831       \prop_if_in:NnT \l__numodel_between_valve_prop {#1}
1832       { \bool_set_true:N \l_tmpb_bool }
1833       \bool_lazy_and:nnTF
1834       { \l__numodel_keep_natural_bool } { \l_tmpb_bool }
1835       { \__numodel_emit_natural_and_phantom:n {#1} }
1836       {
1837         \prop_if_in:NnTF \l__numodel_valve_for_prop {#1}
1838         { \__numodel_emit_valve:n {#1} }
1839         {
1840           \prop_if_in:NnTF \l__numodel_outvalve_for_prop {#1}
1841           { \__numodel_emit_outvalve:n {#1} }
1842           {
1843             \prop_if_in:NnTF \l__numodel_between_valve_prop {#1}
1844             { \__numodel_emit_between_valve:n {#1} }
1845             {
1846               \tl_set:Nc \l__numodel_scratch_tl { \use:c { #1 type } }
1847               \exp_args:NV \str_if_eq:nnT \l__numodel_scratch_tl { stock }
1848               { \__numodel_emit_stock:n {#1} }
1849               \exp_args:NV \str_if_eq:nnT \l__numodel_scratch_tl { aux }
1850               { \__numodel_emit_aux:n {#1} }
1851               \exp_args:NV \str_if_eq:nnT \l__numodel_scratch_tl { constant }
1852               { \__numodel_emit_const:n {#1} }
1853             }
1854           }
1855         }
1856       }
1857     }
1858 }
1859
1860 % --- Helper: emit a coordinate node at the valve position ---
1861 % The flow arrow starts/ends at this coordinate (or at an offset

```

```

1862 % relative to it, for the open-end without cloud). The visible
1863 % white-filled valve is added later (valves_tl) at the same
1864 % coordinates so it overlays the arrow.
1865 % #1 = coord id #2 = x #3 = y
1866 \cs_new_protected:Npn \__numodel_emit_valve_coord:nnn #1 #2 #3
1867 {
1868     \tl_gput_right:Ne \g__numodel_graphic_tl
1869     {
1870         \exp_not:N \coordinate ~
1871         (#1) ~ \exp_not:n{at} ~ (#2 , ~ #3) ;
1872     }
1873 }
1874
1875 % --- Helper: append a valve node to the deferred valves_tl ---
1876 % Renders \node[<valve-style>] (id) at (x,y) {<label>}; later in the
1877 % picture so the white fill overlays the flow arrow underneath.
1878 % #1 = node id #2 = x coord #3 = y coord #4 = label tl name
1879 \cs_new_protected:Npn \__numodel_defer_valve:nnnn #1 #2 #3 #4
1880 {
1881     \tl_put_right:Ne \l__numodel_valves_tl
1882     {
1883         \exp_not:N \node ~ [ \tl_use:N \l__numodel_valve_node_tl ] ~
1884         (#1) ~ \exp_not:n{at} ~ (#2 , ~ #3) ~
1885         {
1886             \bool_if:NTF \l__numodel_valve_label_bool
1887             { \exp_not:N $ \exp_not:V #4 \exp_not:N $ }
1888             { }
1889         } ;
1890     }
1891 }
1892
1893 % --- Forrester/edu-mode emission: natural node + phantom valve ---
1894 % The variable sits as an ordinary aux/const node at its own gridy.
1895 % The phantom valve (id #1__v) sits next to the stock at gridy=0; its
1896 % appearance follows the resolved valve-style (label included only
1897 % when valve-style=edu). A causal arrow from #1 -> #1__v is drawn
1898 % (deferred to late_causals_tl so it lands on top of the white-
1899 % filled valve).
1900 \tl_new:N \l__numodel_phantom_label_tl
1901 \cs_new_protected:Npn \__numodel_emit_natural_and_phantom:n #1
1902 {
1903     % --- 1. Natural node (aux or constant) ---
1904     \tl_set:Ne \l__numodel_scratch_tl { \use:c { #1 type } }
1905     \exp_args:NV \str_if_eq:nnT \l__numodel_scratch_tl { aux }
1906     { \__numodel_emit_aux:n {#1} }
1907     \exp_args:NV \str_if_eq:nnT \l__numodel_scratch_tl { constant }
1908     { \__numodel_emit_const:n {#1} }
1909     % --- 2. Locate the phantom valve x/y position ---
1910     \prop_get:NnNT \l__numodel_vpos_x_prop {#1} \l__numodel_phantom_x_tl
1911     {
1912         \prop_get:NnNF \l__numodel_vpos_y_prop {#1} \l__numodel_scratch_y_tl
1913         { \tl_set:Nn \l__numodel_scratch_y_tl { 0 } }
1914         \tl_set:Ne \l__numodel_phantom_label_tl { \use:c { #1 text } }
1915         % --- 3. Coordinate at the phantom valve position ---
1916         \__numodel_emit_valve_coord:nnn { #1 __vc }
1917         { \tl_use:N \l__numodel_phantom_x_tl }
1918         { \tl_use:N \l__numodel_scratch_y_tl }
1919         % --- 4. Flow arrow + (optional) cloud, one segment ---
1920         \prop_if_in:NnTF \l__numodel_valve_for_prop {#1}
1921         {
1922             \tl_set:Ne \l__numodel_tmp_tl
1923             { \prop_item:Nn \l__numodel_valve_for_prop {#1} }
1924             \__numodel_eff_flowcloud:V \l__numodel_tmp_tl
1925             \str_if_eq:VnTF \l__numodel_flowcloud_eff_tl { true }
1926             {
1927                 \tl_put_right:Ne \l__numodel_flows_tl

```

```

1928         {
1929             \exp_not:N \flowarrow [#1 __cl] {#1 __vc}
1930             { \tl_use:N \l__numodel_tmp_tl }
1931         }
1932     }
1933     {
1934         \tl_put_right:Ne \l__numodel_flows_tl
1935         {
1936             \exp_not:N \flowarrow {#1 __vc}
1937             { \tl_use:N \l__numodel_tmp_tl }
1938         }
1939     }
1940 }
1941 {
1942     \prop_if_in:NnTF \l__numodel_outvalve_for_prop {#1}
1943     {
1944         \tl_set:Ne \l__numodel_tmp_tl
1945         { \prop_item:Nn \l__numodel_outvalve_for_prop {#1} }
1946         \__numodel_eff_flowcloud:V \l__numodel_tmp_tl
1947         \str_if_eq:VnTF \l__numodel_flowcloud_eff_tl { true }
1948         {
1949             \tl_put_right:Ne \l__numodel_flows_tl
1950             {
1951                 \exp_not:N \flowoutarrow [#1 __cl]
1952                 { \tl_use:N \l__numodel_tmp_tl } {#1 __vc}
1953             }
1954         }
1955         {
1956             \tl_put_right:Ne \l__numodel_flows_tl
1957             {
1958                 \exp_not:N \flowoutarrow
1959                 { \tl_use:N \l__numodel_tmp_tl } {#1 __vc}
1960             }
1961         }
1962     }
1963     {
1964         % between (no cloud: both ends are stocks)
1965         \tl_set:Ne \l__numodel_tmp_tl
1966         { \prop_item:Nn \l__numodel_between_valve_prop {#1} }
1967         \tl_set:Ne \l__numodel_scratch_tl
1968         { \prop_item:Nn \l__numodel_between_target_prop {#1} }
1969         \tl_put_right:Ne \l__numodel_flows_tl
1970         {
1971             \exp_not:N \flowbetweenarrow
1972             { \tl_use:N \l__numodel_tmp_tl }
1973             {#1 __vc}
1974             { \tl_use:N \l__numodel_scratch_tl }
1975         }
1976     }
1977 }
1978 % --- 5. Defer the phantom valve drawing (overlays flow) ---
1979 \__numodel_defer_valve:nnnn { #1 __v }
1980 { \tl_use:N \l__numodel_phantom_x_tl }
1981 { \tl_use:N \l__numodel_scratch_y_tl }
1982 \l__numodel_phantom_label_tl
1983 % --- 6. Causal arrow natural -> phantom valve (deferred) ---
1984 \tl_put_right:Ne \l__numodel_late_causals_tl
1985 {
1986     \exp_not:N \draw ~ [\exp_not:n{causal}] ~
1987     (#1) ~ \exp_not:n{to} ~ (#1 __v) ;
1988 }
1989 }
1990 }
1991 \tl_new:N \l__numodel_phantom_x_tl
1992 \cs_generate_variant:Nn \__numodel_eff_flowcloud:n { V }
1993

```



```

1994 \cs_new_protected:Npn \__numodel_emit_stock:n #1
1995 {
1996   \tl_gput_right:Ne \g__numodel_graphic_tl
1997   {
1998     \exp_not:N \node ~ [\exp_not:n{stock}] ~
1999     (#1) ~ \exp_not:n{at} ~
2000     ( \use:c{#1 gridx} , ~ \use:c{#1 gridy} ) ~
2001     { \exp_not:N $ \exp_not:V \l__numodel_tmp_tl \exp_not:N $ } ;
2002   }
2003 }
2004
2005 % Tight-mode helpers: the valve sits at the variable's own gridx/
2006 % gridy. We emit a coordinate node (#1_vc) early so the flow
2007 % arrow has a positioning anchor, and defer the visible valve node
2008 % (#1) to valves_tl so it overlays the arrow with white fill.
2009 \cs_new_protected:Npn \__numodel_emit_tight_valve_setup:n #1
2010 {
2011   \__numodel_emit_valve_coord:nnn { #1 __vc }
2012   { \use:c{#1 gridx} } { \use:c{#1 gridy} }
2013   \__numodel_defer_valve:nnnn {#1}
2014   { \use:c{#1 gridx} } { \use:c{#1 gridy} }
2015   \l__numodel_tmp_tl
2016 }
2017
2018 \cs_new_protected:Npn \__numodel_emit_valve:n #1
2019 {
2020   \__numodel_emit_tight_valve_setup:n {#1}
2021   \tl_set:Ne \l__numodel_scratch_tl
2022   { \prop_item:Nn \l__numodel_valve_for_prop {#1} }
2023   \__numodel_eff_flowcloud:V \l__numodel_scratch_tl
2024   \str_if_eq:VnTF \l__numodel_flowcloud_eff_tl { true }
2025   {
2026     \tl_put_right:Ne \l__numodel_flows_tl
2027     {
2028       \exp_not:N \flowarrow [#1 __cl] {#1 __vc}
2029       { \tl_use:N \l__numodel_scratch_tl }
2030     }
2031   }
2032   {
2033     \tl_put_right:Ne \l__numodel_flows_tl
2034     {
2035       \exp_not:N \flowarrow {#1 __vc}
2036       { \tl_use:N \l__numodel_scratch_tl }
2037     }
2038   }
2039 }
2040
2041 \cs_new_protected:Npn \__numodel_emit_outvalve:n #1
2042 {
2043   \__numodel_emit_tight_valve_setup:n {#1}
2044   \tl_set:Ne \l__numodel_scratch_tl
2045   { \prop_item:Nn \l__numodel_outvalve_for_prop {#1} }
2046   \__numodel_eff_flowcloud:V \l__numodel_scratch_tl
2047   \str_if_eq:VnTF \l__numodel_flowcloud_eff_tl { true }
2048   {
2049     \tl_put_right:Ne \l__numodel_flows_tl
2050     {
2051       \exp_not:N \flowoutarrow [#1 __cl]
2052       { \tl_use:N \l__numodel_scratch_tl } {#1 __vc}
2053     }
2054   }
2055   {
2056     \tl_put_right:Ne \l__numodel_flows_tl
2057     {
2058       \exp_not:N \flowoutarrow
2059       { \tl_use:N \l__numodel_scratch_tl } {#1 __vc}

```

```

2060     }
2061 }
2062 }
2063
2064 \cs_new_protected:Npn \__numodel_emit_between_valve:n #1
2065 {
2066     \__numodel_emit_tight_valve_setup:n {#1}
2067     \tl_set:Nx \l__numodel_scratch_tl
2068     { \prop_item:Nn \l__numodel_between_valve_prop {#1} }
2069     \tl_set:Nx \l__numodel_tmp_tl
2070     { \prop_item:Nn \l__numodel_between_target_prop {#1} }
2071     \tl_put_right:Nx \l__numodel_flows_tl
2072     {
2073         \exp_not:N \flowbetweenarrow
2074         { \tl_use:N \l__numodel_scratch_tl }
2075         {#1 __vc}
2076         { \tl_use:N \l__numodel_tmp_tl }
2077     }
2078 }
2079
2080 \cs_new_protected:Npn \__numodel_emit_aux:n #1
2081 {
2082     \tl_gput_right:Nx \g__numodel_graphic_tl
2083     {
2084         \exp_not:N \node ~ [\exp_not:n{aux}] ~
2085         (#1) ~ \exp_not:n{at} ~
2086         ( \use:c{#1 gridx} , ~ \use:c{#1 gridy} ) ~
2087         { \exp_not:N $ \exp_not:V \l__numodel_tmp_tl \exp_not:N $ } ;
2088     }
2089 }
2090
2091 \cs_new_protected:Npn \__numodel_emit_const:n #1
2092 {
2093     \tl_gput_right:Nx \g__numodel_graphic_tl
2094     {
2095         \exp_not:N \constnode
2096         {#1}
2097         { \use:c{#1 gridx} , ~ \use:c{#1 gridy} }
2098         { \exp_not:N $ \exp_not:V \l__numodel_tmp_tl \exp_not:N $ }
2099     }
2100 }
2101
2102 % --- Stock-as-flow valve emission ---
2103 \cs_new_protected:Npn \__numodel_emit_stock_valve:nn #1 #2
2104 {
2105     % #1 = target stock (e.g. modH), #2 = source stock (e.g. modV)
2106     % Pull gridx from the saved property
2107     \prop_get:NnNTF \l__numodel_stock_valve_prop { #1 __svgx }
2108     \l__numodel_scratch_tl
2109     {
2110         % Look up the wrap-shifted y-slot; default to 0 (the row that
2111         % the auto layout used before any gridmaxx wrap was applied).
2112         \prop_get:NnNF \l__numodel_stock_valve_prop { #1 __svgy }
2113         \l__numodel_scratch_y_tl
2114         { \tl_set:Nn \l__numodel_scratch_y_tl { 0 } }
2115         % Coordinate at the valve position (used by the flow arrow)
2116         \__numodel_emit_valve_coord:nnn { #1 __svc }
2117         { \tl_use:N \l__numodel_scratch_tl }
2118         { \tl_use:N \l__numodel_scratch_y_tl }
2119         % Flow arrow from source-stock through valve into target-stock
2120         % (no cloud: both ends are stocks). The arrow runs in one
2121         % continuous segment; the visible valve will be drawn ON TOP
2122         % later via valves_tl.
2123         \tl_put_right:Nx \l__numodel_flows_tl
2124         {
2125             \exp_not:N \flowbetweenarrow {#2} {#1 __svc} {#1}

```

```

2126     }
2127     % Defer the visible valve node (white fill overlays arrow)
2128     \tl_set:Nc \l__numodel_tmp_tl { \use:c { #2 text } }
2129     \__numodel_defer_valve:nnnn { #1 __sv }
2130     { \tl_use:N \l__numodel_scratch_tl }
2131     { \tl_use:N \l__numodel_scratch_y_tl }
2132     \l__numodel_tmp_tl
2133     % Causal arrow source-stock -> valve. Bend it (curved) only
2134     % when the source stock and the valve sit on the same row -
2135     % then the straight causal would coincide with the flow pipe
2136     % and disappear behind it. When they are on different rows
2137     % (e.g. after a gridmaxx wrap) the causal is vertical-ish and
2138     % stays visible without a bend. Deferred so it lands on top
2139     % of the white-filled valve.
2140     \str_if_eq:eeTF { \tl_use:N \l__numodel_scratch_y_tl }
2141     { \use:c { #2 gridy } }
2142     {
2143         \tl_put_right:Ne \l__numodel_late_causals_tl
2144         {
2145             \exp_not:N \draw ~ [\exp_not:n{causal}] ~
2146             (#2) ~ \exp_not:n{to} ~ [\exp_not:n{bend~left=30}] ~ (#1__sv) ;
2147         }
2148     }
2149     {
2150         \tl_put_right:Ne \l__numodel_late_causals_tl
2151         {
2152             \exp_not:N \draw ~ [\exp_not:n{causal}] ~
2153             (#2) ~ \exp_not:n{to} ~ (#1__sv) ;
2154         }
2155     }
2156 }
2157 { } % no gridx found: skip
2158 }
2159
2160 % --- Stock phantom-valve emission (source stock as rate factor) ---
2161 % Source stock acts as a rate factor for the target stock without a
2162 % matching outflow term; render with a cloud-fed inflow valve next to
2163 % the target stock and link the source stock via a causal arrow.
2164 \cs_new_protected:Npn \__numodel_emit_stock_phantom_valve:nn #1 #2
2165 {
2166     % #1 = target stock (e.g. ballY), #2 = source stock (e.g. ballV)
2167     \prop_get:NnNTF \l__numodel_stock_phantom_valve_prop { #1 __svgx }
2168     \l__numodel_scratch_tl
2169     {
2170         % Look up the wrap-shifted y-slot; default to 0.
2171         \prop_get:NnNF \l__numodel_stock_phantom_valve_prop { #1 __svgy }
2172         \l__numodel_scratch_y_tl
2173         { \tl_set:Nn \l__numodel_scratch_y_tl { 0 } }
2174         % Coordinate at the valve position
2175         \__numodel_emit_valve_coord:nnn { #1 __svc }
2176         { \tl_use:N \l__numodel_scratch_tl }
2177         { \tl_use:N \l__numodel_scratch_y_tl }
2178         % Flow arrow: cloud (open end) -> valve -> target stock
2179         \__numodel_eff_flowcloud:n {#1}
2180         \str_if_eq:VnTF \l__numodel_flowcloud_eff_tl { true }
2181         {
2182             \tl_put_right:Ne \l__numodel_flows_tl
2183             {
2184                 \exp_not:N \flowarrow [ #1 __svcl ] { #1 __svc } { #1 }
2185             }
2186         }
2187         {
2188             \tl_put_right:Ne \l__numodel_flows_tl
2189             {
2190                 \exp_not:N \flowarrow { #1 __svc } { #1 }
2191             }

```

```

2192     }
2193     % Defer the visible valve node, labelled with the source stock
2194     \tl_set:Ne \l__numodel_tmp_tl { \use:c { #2 text } }
2195     \__numodel_defer_valve:nnnn { #1 __sv }
2196     { \tl_use:N \l__numodel_scratch_tl }
2197     { \tl_use:N \l__numodel_scratch_y_tl }
2198     \l__numodel_tmp_tl
2199     % Causal arrow source-stock -> valve. Bend (curved) only when
2200     % source and valve sit on the same row - see the equivalent
2201     % comment in \__numodel_emit_stock_valve:nn for the rationale.
2202     \str_if_eq:eeTF { \tl_use:N \l__numodel_scratch_y_tl }
2203         { \use:c { #2 gridy } }
2204     {
2205         \tl_put_right:Ne \l__numodel_late_causals_tl
2206         {
2207             \exp_not:N \draw ~ [\exp_not:n{causal}] ~
2208             (#2) ~ \exp_not:n{to} ~ [\exp_not:n{bend~left=30}] ~ (#1__sv) ;
2209         }
2210     }
2211     {
2212         \tl_put_right:Ne \l__numodel_late_causals_tl
2213         {
2214             \exp_not:N \draw ~ [\exp_not:n{causal}] ~
2215             (#2) ~ \exp_not:n{to} ~ (#1__sv) ;
2216         }
2217     }
2218 }
2219 { } % no gridx found: skip
2220 }
2221
2222 % -----
2223 % Pillar A - Lua-side layout writeback. A single directlua call
2224 % that runs compute_layout (build_deps, classify_flows,
2225 % populate_occupied, auto_layout, causals) and writes the resulting
2226 % state back to the TeX-side props (\<var>gridx/gridy and the
2227 % flow-props) that the downstream emitters (place_node,
2228 % flow-builders, emit_natural_and_phantom, emit_stock_valve) read.
2229 % Lua is the single source of truth for the decision logic; TeX
2230 % only renders.
2231 % -----
2232 \cs_new_protected:Npn \__numodel_lua_layout_writeback:
2233 {
2234     % 1. Clear the props that Lua refills, so repeated \graphicmodel
2235     % calls don't produce duplicate entries.
2236     \prop_clear:N \l__numodel_valve_for_prop
2237     \prop_clear:N \l__numodel_outvalve_for_prop
2238     \prop_clear:N \l__numodel_between_valve_prop
2239     \prop_clear:N \l__numodel_between_target_prop
2240     \prop_clear:N \l__numodel_stock_valve_prop
2241     \prop_clear:N \l__numodel_stock_phantom_valve_prop
2242     \prop_clear:N \l__numodel_vpos_x_prop
2243     \prop_clear:N \l__numodel_vpos_y_prop
2244     % 2. Run the Lua pipeline.
2245     \directlua{ numodel.compute_layout(
2246         "\g_numodel_current_prefix_tl",
2247         "\g__numodel_diagram_style_tl",
2248         \int_use:N \g_numodel_gridmaxx_int) }
2249     % 3. Write gridx/gridy and all flow-props back.
2250     % tex_writeback emits a list of expl3 statements via tex.print.
2251     \directlua{ numodel.tex_writeback(
2252         "\g_numodel_current_prefix_tl") }
2253 }
2254
2255 \cs_new_protected:Npn \__numodel_lua_emit_causals:
2256 {
2257     \directlua{ numodel.emit_causals(

```

```

2258     "\g_numodel_current_prefix_tl") }
2259 }
2260
2261 \NewDocumentCommand{\graphicmodel}{0}{ }{%
2262   \tl_clear:N \l__numodel_cmd_prefix_tl
2263   \tl_clear:N \l__numodel_cmd_diagstyle_tl
2264   \tl_clear:N \l__numodel_cmd_flowarrow_tl
2265   \tl_clear:N \l__numodel_cmd_valve_tl
2266   \tl_clear:N \l__numodel_cmd_flowcloud_tl
2267   \keys_set:nn { numodel / cmd } {#1}
2268   \__numodel_resolve_eff_prefix:
2269   % Temporary overrides: save the global state, apply the per-call
2270   % key values before build, restore afterwards. This way
2271   % \graphicmodel[diagram-style=forrester] flips one render without
2272   % touching the global \numodelsetup state.
2273   \tl_set_eq:NN \l__numodel_saved_diagstyle_tl \g__numodel_diagram_style_tl
2274   \tl_set_eq:NN \l__numodel_saved_flowarrow_tl \g__numodel_flowarrow_style_tl
2275   \tl_set_eq:NN \l__numodel_saved_valve_tl \g__numodel_valve_style_tl
2276   \tl_set_eq:NN \l__numodel_saved_flowcloud_tl \g__numodel_flowcloud_tl
2277   \tl_if_empty:NF \l__numodel_cmd_diagstyle_tl
2278     { \tl_gset_eq:NN \g__numodel_diagram_style_tl \l__numodel_cmd_diagstyle_tl }
2279   \tl_if_empty:NF \l__numodel_cmd_flowarrow_tl
2280     { \tl_gset_eq:NN \g__numodel_flowarrow_style_tl \l__numodel_cmd_flowarrow_tl }
2281   \tl_if_empty:NF \l__numodel_cmd_valve_tl
2282     { \tl_gset_eq:NN \g__numodel_valve_style_tl \l__numodel_cmd_valve_tl }
2283   \tl_if_empty:NF \l__numodel_cmd_flowcloud_tl
2284     { \tl_gset_eq:NN \g__numodel_flowcloud_tl \l__numodel_cmd_flowcloud_tl }
2285   % Resolve the effective values for this render and propagate them
2286   % to \nmflowbody / \nmflowtip used by the flow macros.
2287   \__numodel_resolve_flowarrow:
2288   \__numodel_resolve_valve:
2289   \__numodel_resolve_flowcloud:
2290   \__numodel_apply_flowarrow_style:
2291   \__numodel_apply_valve_style:
2292   \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
2293   {
2294     \__numodel_build_graphic:
2295     \tl_use:N \g__numodel_graphic_tl
2296   }%
2297   \tl_gset_eq:NN \g__numodel_diagram_style_tl \l__numodel_saved_diagstyle_tl
2298   \tl_gset_eq:NN \g__numodel_flowarrow_style_tl \l__numodel_saved_flowarrow_tl
2299   \tl_gset_eq:NN \g__numodel_valve_style_tl \l__numodel_saved_valve_tl
2300   \tl_gset_eq:NN \g__numodel_flowcloud_tl \l__numodel_saved_flowcloud_tl
2301 }
2302 \tl_new:N \l__numodel_saved_diagstyle_tl
2303 \tl_new:N \l__numodel_saved_flowarrow_tl
2304 \tl_new:N \l__numodel_saved_valve_tl
2305 \tl_new:N \l__numodel_saved_flowcloud_tl
2306
2307 % =====
2308 % Plot machinery
2309 % =====
2310 % \diagrammodel calls the pgfplots/calculotdims infrastructure
2311 % through internal wrappers \__numodel_calc_plot_dims: and
2312 % \__numodel_draw_plot:n. Those wrappers forward to \calculotdims
2313 % and \drawplot from the sibling package numodel-plot.
2314 %
2315 % The wrappers are not \cs_new_eq:NN because numodel-plot can load
2316 % after this package; at load time \calculotdims and \drawplot may
2317 % not yet exist. With a wrapper they are looked up at expansion
2318 % time.
2319
2320 \cs_new:Npn \__numodel_calc_plot_dims: { \calculotdims }
2321 \cs_new:Npn \__numodel_draw_plot:n #1 { \drawplot {#1} }
2322
2323 % =====

```

```

2324 % \diagrammodel -- convenience command for model graphs
2325 % =====
2326 % Usage: \diagrammodel{xvar}{yvar}[extra drawplot code]{label}
2327 %
2328 % Builds a complete figure with model points. Sets axis dimensions
2329 % from min/max, axis labels from text/unit, and emits caption and
2330 % label automatically.
2331
2332 \tl_new:N \l__numodel_xname_tl
2333 \tl_new:N \l__numodel_yname_tl
2334
2335 \NewDocumentCommand{\diagrammodel}{ O{} m m O{} m }
2336 {
2337   \tl_clear:N \l__numodel_cmd_prefix_tl
2338   \keys_set:nn { numodel / cmd } {#1}
2339   \__numodel_resolve_eff_prefix:
2340   % Full names (prefix + short) for \use:c lookup on globals
2341   \tl_set:Nx \l__numodel_xname_tl { \l__numodel_eff_prefix_tl #2 }
2342   \tl_set:Nx \l__numodel_yname_tl { \l__numodel_eff_prefix_tl #3 }
2343   \edef\dm@coords{\mcoordsp{\l__numodel_eff_prefix_tl}{#2}{#3}}
2344   \def\xmin{\use:c{\l__numodel_xname_tl min}}
2345   \def\xmax{\use:c{\l__numodel_xname_tl max}}
2346   \def\xlabelqty{\use:c{\l__numodel_xname_tl text}}
2347   \def\xlabelunit{\use:c{\l__numodel_xname_tl unitraw}}
2348   \def\ymin{\use:c{\l__numodel_yname_tl min}}
2349   \def\ymax{\use:c{\l__numodel_yname_tl max}}
2350   \def\ylabelqty{\use:c{\l__numodel_yname_tl text}}
2351   \def\ylabelunit{\use:c{\l__numodel_yname_tl unitraw}}
2352   \begin{figure}[H]
2353     \centering
2354     \group_begin:
2355     \__numodel_apply_dsep:
2356     \__numodel_draw_plot:n
2357     {
2358       \addplot[only~marks,~mark=*,~mark~size=0.5pt]~coordinates~{\dm@coords};
2359       \addlegendentry{model~point}
2360       #4
2361     }
2362     \group_end:
2363     \caption{$\use:c{\l__numodel_yname_tl text}$(\use:c{\l__numodel_xname_tl text})\text{-}
diagram}}
2364     % If the project-specific worksheet system is loaded, pick the
2365     % '-ws' label inside a worksheet; otherwise always the bare label.
2366     \cs_if_exist:NTF \ifinworksheet
2367       { \ifinworksheet{\label{fig:#5-ws}}{\label{fig:#5}} }
2368       { \label{fig:#5} }
2369     \end{figure}
2370   }
2371
2372 \ExplSyntaxOff
2373
2374 % =====
2375 % GRAPHIC MODEL - STYLES AND MACROS
2376 % =====
2377 %
2378 % Forrester-diagram conventions:
2379 %
2380 % Stock [stock] Rectangle v, s, T, x
2381 % Valve / inflow [valve] Circle on a thick a, dv, ds
2382 % flow pipe
2383 % Auxiliary [aux] Plain circle F_res, F_air
2384 % Constant \constnode Circle + dashes m, k, g
2385 %
2386 % Use \dgridx and \dgridy as the grid spacings for consistent positioning.
2387 % =====
2388

```

```

2389 % === Grid spacing -- defaults set via \numodelsetup {graphscalex, graphscaley, stockwidth} ===
2390
2391 % === Styles ===
2392 \tikzset{
2393   % Scale coordinates to the grid spacing
2394   gridscale/.style={x=\dgridx cm, y=\dgridy cm},
2395   % Stock (state variable): rectangle
2396   stock/.style={
2397     rectangle, draw, thick,
2398     minimum width=3 em,
2399     minimum height=2 em,
2400     font=\small
2401   },
2402   % Auxiliary (intermediate variable): plain circle
2403   aux/.style={
2404     circle, draw, thick,
2405     inner sep=2pt,
2406     minimum size=2 em,
2407     font=\scriptsize
2408   },
2409   % Constant: circle (dashes drawn by the \constnode macro)
2410   const/.style={
2411     circle, draw, thick,
2412     inner sep=2pt,
2413     minimum size=2 em,
2414     font=\scriptsize
2415   },
2416   % Valve (legacy alias): circle on the flow pipe. Kept for
2417   % backwards compatibility with user code that calls \flowarrow
2418   % manually. The package's emit helpers now pick one of the
2419   % style variants below based on \numodelsetup{valve-style=...}.
2420   valve/.style={
2421     circle, draw, thick, fill=white,
2422     minimum size=2 em,
2423     font=\small
2424   },
2425   % Valve variant: Forrester valve symbol -- two filled triangles
2426   % (white inside) whose tips meet at the centre. The symmetry
2427   % axis stands perpendicular to the (horizontal) flow. The
2428   % bounding box is tall and narrow so the apex angle at the
2429   % centre is roughly 60 degrees (sharper than the 90 degrees of
2430   % a square box), and the horizontal "ribs" along the top and
2431   % bottom edges close the two triangles.
2432   valve-forrester/.style={
2433     shape=rectangle,
2434     draw=none, fill=none,
2435     inner sep=0pt, outer sep=0pt,
2436     %minimum width=\dgridx*0.18 cm,
2437     minimum width=1 em,
2438     %minimum height=\dgridy*0.30 cm,
2439     minimum height=2 em,
2440     path picture={%
2441       \draw[thick, fill=white]
2442         (path picture bounding box.north west) --
2443         (path picture bounding box.north east) --
2444         (path picture bounding box.center) --
2445         cycle;
2446       \draw[thick, fill=white]
2447         (path picture bounding box.south west) --
2448         (path picture bounding box.south east) --
2449         (path picture bounding box.center) --
2450         cycle;
2451     }
2452   },
2453   % Valve variant: empty circle (no label).
2454   valve-circle/.style={

```

```

2455 circle, draw, thick, fill=white,
2456 minimum size=2 em,
2457 font=\small
2458 },
2459 % Valve variant: circle with the variable's display label inside.
2460 valve-edu/.style={
2461 circle, draw, thick, fill=white,
2462 minimum size=2 em,
2463 font=\small
2464 },
2465 % Flow pipe variants (body without arrow tip, used for the
2466 % segment from open-end / cloud to the valve):
2467 flowpipe/.style={line width=3pt}, % legacy alias
2468 flowpipe-filled/.style={line width=3pt},
2469 flowpipe-hollow/.style={
2470 line width=1pt,
2471 double=white, double distance=3pt
2472 },
2473 % Flow pipe variants WITH arrow tip (used for the segment from
2474 % valve to stock, or the open-end side of an outflow). Both
2475 % variants use the Stealth arrow tip. The hollow form
2476 % uses the double-line magic length 0pt-3-0 so that the arrow
2477 % tip merges cleanly with the gap between the two strokes.
2478 flowpipe-filled-tip/.style={line width=3pt, arrows={-Stealth[inset=0pt, angle=30:1em]}},
2479 flowpipe-hollow-tip/.style={
2480 line width=1pt,
2481 double=white, double distance=3pt,
2482 arrows={-Stealth[inset=0pt, angle=30:1em]}
2483 },
2484 % Flow pipe variants with the Cloud arrow tip on the open end:
2485 % inflow keeps the Stealth arrow head at the stock side.
2486 flowpipe-filled-cloudin/.style={
2487 line width=3pt,
2488 arrows={Cloud[fill=white, line width=0.8pt]-Stealth[inset=0pt, angle=30:1em]}
2489 },
2490 flowpipe-hollow-cloudin/.style={
2491 line width=1pt,
2492 double=white, double distance=3pt,
2493 arrows={Cloud[fill=white, line width=0.8pt]-Stealth[inset=0pt, angle=30:1em]}
2494 },
2495 % Outflow with cloud at the open end: first a Stealth tip at the
2496 % line end, then the cloud behind it (in arrows.meta spec, list
2497 % multiple tips separated by a dot so the shaft stops at the
2498 % Stealth tip and does not extend into the cloud; order = from
2499 % line outwards, hence Stealth.Cloud).
2500 flowpipe-filled-cloudout/.style={
2501 line width=3pt,
2502 arrows={-{Stealth[inset=0pt, angle=30:1em].Cloud[fill=white, line width=0.8pt]}}
2503 },
2504 flowpipe-hollow-cloudout/.style={
2505 line width=1pt,
2506 double=white, double distance=3pt,
2507 arrows={-{Stealth[inset=0pt, angle=30:1em].Cloud[fill=white, line width=0.8pt]}}
2508 },
2509 % Causal arrow (thin arrow for dependencies)
2510 causal/.style={thick, arrows={-Stealth[length=0.5em]}}
2511 }
2512
2513 % Default values for the body / tip macros so a manual call to
2514 % \flowarrow (outside \graphicmodel) keeps working. The
2515 % \graphicmodel pipeline overwrites these per render.
2516 \providecommand{\nmflowbody}{flowpipe-filled}
2517 \providecommand{\nmflowtip}{flowpipe-filled-tip}
2518 \providecommand{\nmflowtipcloudin}{flowpipe-filled-cloudin}
2519 \providecommand{\nmflowtipcloudout}{flowpipe-filled-cloudout}
2520

```



```

2521 % === Macro: constant node with dashes ===
2522 % Usage: \constnode{name}{(x,y)}{$label$}
2523 \newcommand{\constnode}[3]{%
2524 \node[const] (#1) at (#2) {#3};%
2525 \draw[thick] ([xshift=-3mm]#1.west) -- (#1.west);%
2526 \draw[thick] (#1.east) -- ([xshift=3mm]#1.east);%
2527 }
2528
2529 % === Macro: flow pipe with valve (inflow) ===
2530 % Draws ONE continuous flow arrow ending at the stock. The starting
2531 % position is the same regardless of cloud presence; passing any
2532 % non-blank optional argument switches to the cloud-tipped variant
2533 % so the arrow's tail becomes a white-filled cloud (drawn as a real
2534 % PGF arrow tip, not a separate node - moves with the line). The
2535 % valve overlays the middle of the arrow as a separate white-filled
2536 % node drawn later by the build pipeline.
2537 % Style keys: \nmflowtip / \nmflowtipcloudin.
2538 % Usage: \flowarrow[<any>]{<valve-coord>}{<stock-node>}
2539 \NewDocumentCommand{\flowarrow}{ O{} m m }{%
2540 \IfBlankTF{#1}{%
2541 \draw[\nmflowtip] ([xshift=-3em]#2) -- ([xshift=0.5mm]#3.west);%
2542 }{%
2543 \draw[\nmflowtipcloudin] ([xshift=-3em]#2) -- ([xshift=0.5mm]#3.west);%
2544 }%
2545 }
2546
2547 % === Macro: flow pipe with valve (outflow) ===
2548 % Mirror of \flowarrow. The arrow always ends at the same offset
2549 % past the valve coordinate; passing a non-blank optional argument
2550 % switches to the cloud-tipped variant (cloud at the open end).
2551 % Usage: \flowoutarrow[<any>]{<stock-node>}{<valve-coord>}
2552 \NewDocumentCommand{\flowoutarrow}{ O{} m m }{%
2553 \IfBlankTF{#1}{%
2554 \draw[\nmflowtip] (#2.east) -- ([xshift=4em]#3);%
2555 }{%
2556 \draw[\nmflowtipcloudout] (#2.east) -- ([xshift=5em]#3);%
2557 }%
2558 }
2559
2560 % === Macro: flow pipe between two stocks ===
2561 % One continuous arrow from source stock through valve coord into
2562 % target stock. No cloud option (no open end exists when both
2563 % sides are stocks).
2564 % Usage: \flowbetweenarrow[<source-stock>]{<valve-coord>}{<target-stock>}
2565 \newcommand{\flowbetweenarrow}[3]{%
2566 \draw[\nmflowtip] (#1.east) -- ([xshift=0.5mm]#3.west);%
2567 }
2568 \end{package}

```

9.1 Translation files

Each `numodel-<LANG>.def` file populates the lookup table consulted by `\__numodel_kw:n`. The package loads exactly the file matching the current `syntax` key, via `\InputIfFileExists` (and hence `kpse`), so additional languages can be supplied through `TEXMFHOME/tex/latex/numodel/numodel-<LANG>.def` with no package rebuild. The two shipped files follow.

9.1.1 numodel-EN.def – English (XMILE)

XMILE v1.0 OASIS style: ALL-CAPS, case sensitive. Sets the default decimal mark to point.

```

2569 (*EN)
2570 \ProvidesExplFile{numodel-EN.def}{2026/05/17}{0.3.0}
2571 {numodel-keyword-table~---English~(XMILE)}
2572 \cs_new:Npn \__numodel_kw_EN_if: { IF~ }
2573 \cs_new:Npn \__numodel_kw_EN_then: { ~THEN~ }
2574 \cs_new:Npn \__numodel_kw_EN_then_nl: { ~THEN~ }
2575 \cs_new:Npn \__numodel_kw_EN_else: { ~ELSE~ }

```

```

2576 \cs_new:Npn \__numodel_kw_EN_else_nl: { ELSE }
2577 \cs_new:Npn \__numodel_kw_EN_endif: { ~ENDIF }
2578 \cs_new:Npn \__numodel_kw_EN_endif_nl: { ENDIF }
2579 \cs_new:Npn \__numodel_kw_EN_and: { ~AND~ }
2580 \cs_new:Npn \__numodel_kw_EN_or: { ~OR~ }
2581 \cs_new:Npn \__numodel_kw_EN_not: { NOT~ }
2582 \cs_new:Npn \__numodel_kw_EN_sign: { SIGN }
2583 \cs_new:Npn \__numodel_kw_EN_abs: { ABS }
2584 \cs_new:Npn \__numodel_kw_EN_sqrt: { SQRT }
2585 \cs_new:Npn \__numodel_kw_EN_exp: { EXP }
2586 \cs_new:Npn \__numodel_kw_EN_ln: { LN }
2587 \cs_new:Npn \__numodel_kw_EN_sin: { SIN }
2588 \cs_new:Npn \__numodel_kw_EN_cos: { COS }
2589 \cs_new:Npn \__numodel_kw_EN_tan: { TAN }
2590 \cs_new:Npn \__numodel_kw_EN_asin: { ARCSIN }
2591 \cs_new:Npn \__numodel_kw_EN_acos: { ARCCOS }
2592 \cs_new:Npn \__numodel_kw_EN_stop: { ~STOP~ }
2593 \cs_new:Npn \__numodel_kw_EN_th_model: { model }
2594 \cs_new:Npn \__numodel_kw_EN_th_initvals: { initial~values }
2595 \cs_new:Npn \__numodel_kw_EN_dsep_default: { point }
2596 </EN>

```

9.1.2 numodel-NL.def – Dutch (CoachTaal)

CoachTaal convention. Sets the default decimal mark to comma.

```

2597 (*NL)
2598 \ProvidesExplFile{numodel-NL.def}{2026/05/17}{0.3.0}
2599 {numodel~keyword~table~---Dutch~(CoachTaal)}
2600 \cs_new:Npn \__numodel_kw_NL_if: { Als~ }
2601 \cs_new:Npn \__numodel_kw_NL_then: { ~Dan~ }
2602 \cs_new:Npn \__numodel_kw_NL_then_nl: { ~Dan }
2603 \cs_new:Npn \__numodel_kw_NL_else: { ~Anders~ }
2604 \cs_new:Npn \__numodel_kw_NL_else_nl: { Anders }
2605 \cs_new:Npn \__numodel_kw_NL_endif: { ~EindAls }
2606 \cs_new:Npn \__numodel_kw_NL_endif_nl: { EindAls }
2607 \cs_new:Npn \__numodel_kw_NL_and: { ~EN~ }
2608 \cs_new:Npn \__numodel_kw_NL_or: { ~OF~ }
2609 \cs_new:Npn \__numodel_kw_NL_not: { NIET~ }
2610 \cs_new:Npn \__numodel_kw_NL_sign: { Teken }
2611 \cs_new:Npn \__numodel_kw_NL_abs: { Abs }
2612 \cs_new:Npn \__numodel_kw_NL_sqrt: { Sqrt }
2613 \cs_new:Npn \__numodel_kw_NL_exp: { Exp }
2614 \cs_new:Npn \__numodel_kw_NL_ln: { Ln }
2615 \cs_new:Npn \__numodel_kw_NL_sin: { Sin }
2616 \cs_new:Npn \__numodel_kw_NL_cos: { Cos }
2617 \cs_new:Npn \__numodel_kw_NL_tan: { Tan }
2618 \cs_new:Npn \__numodel_kw_NL_asin: { Arcsin }
2619 \cs_new:Npn \__numodel_kw_NL_acos: { Arccos }
2620 \cs_new:Npn \__numodel_kw_NL_stop: { ~Stop~ }
2621 \cs_new:Npn \__numodel_kw_NL_th_model: { model }
2622 \cs_new:Npn \__numodel_kw_NL_th_initvals: { startwaarden }
2623 \cs_new:Npn \__numodel_kw_NL_dsep_default: { comma }
2624 </NL>

```